

2022 - 2023

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

**SPÉCIALITÉ : Architecture IT et Cloud
Computing**

**Implémentation d'un Framework pour
l'automatisation des tests Et mise en place d'une
chaîne CI/CD avec Azure Devops**

Réalisé par: Hazar Agili

Encadré par:

Encadrant ESPRIT: Mr Wael doghri

Encadrant Entreprise: Mr Rafik Dahmen

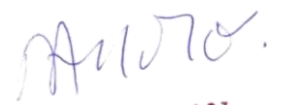


Tesson Group

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Rafik Dahmen**

Signature et cachet


TESSAN
17, Rue Ahmed Rami
1002 Tunis
MF: 1592662 H / A / M / 000

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Wael Doghri**

Signature

Dédicaces

Je dédie ce travail à ma famille, surtout à mes parents **Zina** et **Aidi**, dont le soutien et l'amour inconditionnels m'ont donné la force et la motivation nécessaires pour accomplir ce projet. Leur présence constante à mes côtés m'a été d'un grand réconfort, et leur confiance en moi m'a poussé à donner le meilleur de moi-même.

À mes amis, pour leur compréhension et leur soutien moral inestimable. Leur présence et leurs encouragements ont rendu ce parcours plus supportable et ont souvent égayé mes journées de travail intense.

À mes collègues, pour leur collaboration et leur aide précieuse tout au long de ce parcours. Leur esprit d'équipe et leur camaraderie ont été essentiels à la réalisation de ce projet.

Remerciements

Je tiens à exprimer mes plus sincères remerciements à **Monsieur Rafik Dahmen**, mon encadrant professionnel au sein de TESSAN GROUP, pour son dévouement, son soutien constant et ses précieux conseils tout au long de la réalisation de ce projet. Sa guidance experte et sa patience ont été d'une aide inestimable, et j'ai énormément apprécié chaque moment de collaboration avec lui.

Je suis également profondément reconnaissant envers **Monsieur Doghri Wael**, mon encadrant pédagogique à l'École Supérieure Privée d'Ingénierie et de Technologie (ESPRIT), pour son encadrement attentif, ses encouragements et son expertise académique. Ses conseils éclairés et son soutien ont été essentiels pour mener à bien ce travail de recherche.

Mes remerciements s'adressent également à tous les membres du jury pour avoir accepté d'évaluer ce travail. Leur expertise et leurs retours ont été extrêmement précieux pour m'aider à progresser dans ma démarche académique.

Enfin, je souhaite exprimer toute ma gratitude envers chacun de mes enseignants à **l'École Supérieure Privée de Technologie et d'Ingénierie (ESPRIT)**. Leur dévouement à transmettre leur savoir, leur passion pour l'enseignement et leur engagement envers leurs étudiants ont joué un rôle déterminant dans mon parcours académique et professionnel.

Je suis profondément reconnaissant envers toutes ces personnes qui ont contribué de près ou de loin à la réalisation de ce projet, et je leur suis infiniment reconnaissant pour leur soutien et leur confiance en moi.

Table des matières

Introduction générale	1
1 Cadre général du projet	3
Introduction	4
1.1 Présentation de l'organisme d'accueil	4
1.2 Présentation du projet	5
1.2.1 Etude de l'existant	5
1.2.2 Problématique	6
1.2.3 Solution Proposée	7
1.2.4 Objectifs	8
1.3 Méthodologie adaptée	8
Conclusion	9
2 Étude de de faisabilité et Architecture	10
Introduction	11
2.1 Identification des acteurs du système	11
2.2 Identification des besoins	11
2.2.1 Besoins fonctionnels	11
2.2.2 Besoins non fonctionnels	12
2.3 Backlog produit	13
2.4 Architecture globale de la solution	14
2.4.1 Architecture	14
2.4.2 Les Différentes Couches du framowrk	16
Conclusion	18
3 Étude et réalisation du sprint 1 : Implémentation du Framework	19
Introduction	20
3.1 Backlog du sprint	20
3.2 Choix technologiques	20
3.2.1 Selenium	20

3.2.2	C sharp :	21
3.2.3	.NET Core	21
3.2.4	Extent Reports :	22
3.2.5	JSON.NET :	22
3.2.6	Excel Data Reader :	23
3.2.7	HttpClient :	23
3.2.8	Azure DevOps :	24
3.3	Réalisation	24
3.3.1	Implémentation du CORE	24
3.3.2	Configuration des Paramètres du Gestionnaire de Packages dans Visual Studio	26
	Conclusion	26
4	Étude et réalisation du sprint 2 : Développement des scénarios de tests	27
	Introduction	28
4.1	Backlog du sprint	28
4.2	Choix technologiques	29
4.2.1	NUnit :	29
4.3	Présentation du processus de tests	29
4.4	Réalisation	30
4.4.1	Identification des besoins métier :	31
4.4.2	Regroupement des scénarios de test :	32
4.4.3	Évaluation des scénarios existants :	33
4.4.4	Développement de nouveaux scénarios :	33
4.4.5	Gestion des données d'entrée :	34
4.4.6	Définition du workflow des tests :	35
	Conclusion	37
5	Étude et réalisation du sprint 3 : Mise en place d'un pipeline CI/CD dans Azure DevOps	38
	Introduction	39
5.1	Backlog du sprint	39
5.2	Choix technologiques	39

5.2.1	Azure DevOps :	39
5.3	Architecture de l'environnement	41
5.4	Réalisation	42
5.4.1	Intégration de Visual Studio Code avec Azure Repos	43
5.4.2	Création d'une nouvelle pipeline CI/CD	44
5.4.3	Configuration des Étapes de Build dans Azure DevOps	44
5.4.4	Configuration des Artefacts dans la Pipeline CI/CD Azure DevOps	45
	Conclusion	48
6	Étude et réalisation du sprint 4 : L'exécution des tests avec QC ALM	50
	Introduction	51
6.1	Backlog du sprint	51
6.2	Choix technologiques	51
6.2.1	Application Lifecycle Management(ALM)	51
6.3	Architecture de l'environnement	53
6.4	Réalisation	54
6.4.1	Implémentation du Déploiement d'Artefacts dans ALM et Exécution des Tests avec Rapports d'Analyse	54
	Conclusion	58
	Conclusion générale	59

Table des figures

1.1	Logo De Tessan	4
1.2	logo de l'UFT	6
2.1	Architecture de la solution	15
2.2	Les Différentes Couches du framowrk	16
3.1	Selenium	21
3.2	C sharp	21
3.3	.NET Core	22
3.4	Extent Reports	22
3.5	JSON.NET	23
3.6	Excel Data Reader	23
3.7	HttpClient	24
3.8	Azure DevOps	24
3.9	Exploration des Packages NuGet dans Azure DevOps pour une Intégration Fluide dans Visual Studio Code	25
3.10	Configuration des Paramètres du Gestionnaire de Packages	26
4.1	Nuint	29
4.2	Phase de définition des tests	30
4.3	Capture d'écran de l'application testée	31
4.4	un exemple de scénario de test	33
4.5	un exemple de fichier de données	35
4.6	un exemple de fichier workflow	36
4.7	les différentes couches utiliser pour exécuter le test	37
5.1	Azure DevOps	40
5.2	L'architecture de notre solution avec Azure DevOps	42
5.3	Interface de Azure Devops	43
5.4	le processus pour la mise en place d'une pipeline CI/CD	43
5.5	Étape de la pipeline de build	46

5.6	les étapes de copie, d'archivage et de publication	46
5.7	La figure représente l'exécution de la pipeline CI/CD dans Azure DevOps,	48
5.8	les artefacts générés	48
6.1	logo de l'ALM	52
6.2	Ressources de test	52
6.3	la section des Tests	53
6.4	l'architecture de Workflow	53
6.5	Artefacts(packages et workflow)	55
6.6	La configuration du script de type VAPI XP	55
6.7	la configuration appropriée à la plante	56
6.8	Rapport suite à l'exécution réussie du test	57
6.9	Rapport suite à l'exécution infructueuse du test	58

Liste des tableaux

- 1.1 la comparaison entre les deux méthode agile et classique 9
- 2.1 Backlog du produit 14
- 3.1 Backlog du sprint 1 20
- 4.1 Backlog du sprint 2 28
- 5.1 Backlog du Sprint 3 39
- 5.2 Comparaison entre Azure Devops et Jenkins 40
- 6.1 Backlog du sprint 4 51

Liste des abréviations

ISTQB :International Software Testing Qualification Board

QC ALM : Quality Center Application Life Cycle Management

CI/CD : Continuous integration/continuous delivery

UFT : Unified Functional Testing

Introduction générale

L'évolution rapide du monde du développement logiciel a engendré des changements significatifs dans les approches traditionnelles de conception, de développement, et de gestion de logiciels. La demande croissante pour des applications robustes, évolutives et de haute qualité a conduit à l'essor de l'automatisation des processus liés au cycle de vie du logiciel. Cette transformation est motivée par divers facteurs, tous convergeant vers un objectif commun : améliorer l'efficacité, la qualité, et la rapidité des projets de développement. Dans cette perspective, l'automatisation s'est imposée comme un pilier incontournable du développement logiciel moderne.

Le contexte actuel du développement logiciel est caractérisé par la nécessité impérieuse d'atteindre des objectifs tels que la rapidité de livraison, la qualité du code, l'efficacité opérationnelle, et l'adaptabilité aux changements constants du marché. Face à ces enjeux, l'automatisation se présente comme une solution essentielle, permettant d'accélérer les cycles de développement, de garantir la qualité du code par des tests automatisés, d'optimiser les processus opérationnels, et de favoriser la flexibilité et l'adaptabilité aux évolutions technologiques.

Les objectifs fondamentaux de l'automatisation dans le développement logiciel incluent l'optimisation des processus, la réduction des coûts, l'amélioration de la qualité, et l'alignement plus étroit sur les exigences métier. En réduisant les erreurs humaines, en minimisant les coûts associés aux erreurs de développement, et en assurant une satisfaction client accrue, l'automatisation devient un catalyseur essentiel de la réussite des projets logiciels.

Dans cette optique, nous structurerons notre projet selon le cadre du processus Scrum. À cette fin, le rapport illustre les différentes phases de réalisation du projet, qui se divisent en six chapitres :

- Dans le premier chapitre, nous présenterons l'entreprise Tessan, exposerons la problématique existante, mettrons l'accent sur la solution proposée et aborderons la méthode adoptée.
- Le deuxième chapitre décrira les différents besoins fonctionnels et non fonctionnels, le backlog de produit, ainsi que l'architecture générale.
- Le troisième chapitre présentera le premier sprint, qui concerne l'implémentation du framework de tests.
- Dans le quatrième chapitre, nous présenterons le deuxième sprint, qui développe des scénarios de

tests.

- Le cinquième chapitre abordera la mise en place d'un pipeline CI/CD avec Azure DevOps.
- Enfin, dans le sixième chapitre, nous clôturerons par l'exécution des tests sur ALM, une application qui aide à gérer les activités de gestion du cycle de vie des applications et à mettre en œuvre des processus rigoureux et auditable.

Nous concluons par une conclusion générale des forces et des faiblesses de notre effort.

CADRE GÉNÉRAL DU PROJET

Plan

Introduction	4
1 Présentation de l'organisme d'accueil	4
2 Présentation du projet	5
3 Méthodologie adaptée	8
Conclusion	9

Introduction

Dans ce chapitre, nous présentons d'abord l'organisation d'accueil, puis nous exposons le cadre général de notre projet. Nous présentons ensuite la problématique que nous souhaitons résoudre et la solution que nous souhaitons mettre en œuvre. Cela nous amènera à définir la méthodologie de travail adoptée dans notre projet.

1.1 Présentation de l'organisme d'accueil

Tessan Group est une entreprise de services logiciels agile et cloud qui s'adresse à des clients internationaux. L'entreprise adopte la culture DevTestOps et valorise l'approche agile, mettant l'accent sur l'itération rapide et les retours continus. L'organisation attache également une grande importance à l'excellence technique, considérant que cela joue un rôle essentiel dans le maintien de son dynamisme.

[1] Les services offerts par Tessan Group comprennent :

- **Tessan Technologie** : notre offre de services pour DevTestOps avec les technologies Microsoft et le cloud Azure.
- **Tessan Testing** : notre offre de services pour les tests logiciels basée sur la norme internationale ISTQB.
- **Tessan Talent** : notre service pour assister les clients dans la gestion des talents.



FIGURE 1.1 : Logo De Tessan

1.2 Présentation du projet

Le projet que nous entreprenons vise à implémenter un framework de tests automatisés pour évaluer et améliorer la robustesse des scénarios de test de l'application. Face aux limites des outils précédemment utilisés, nous cherchons à développer une solution plus flexible et agile. En parallèle, nous renforçons notre expertise par des certifications et intégrons des pratiques modernes comme CI/CD pour optimiser nos processus de test. Cette initiative représente une avancée significative vers une gestion plus efficace et de meilleure qualité des tests logiciels.

1.2.1 Etude de l'existant

Micro Focus UFT

Unified Functional Testing (UFT) a longtemps été une référence dans le domaine de l'automatisation des tests. Cet outil, également connu sous le nom de HPE UFT (anciennement HP QuickTest Professional), a joué un rôle central dans la simplification des processus de test fonctionnels, en particulier dans des environnements tels que les applications web et bureautiques..

UFT a été salué pour sa facilité d'utilisation, sa compatibilité étendue et sa prise en charge de diverses technologies. Cependant, il présente également certaines limites :

Adaptation aux technologies émergentes : UFT a du mal à s'adapter aux nouvelles technologies et aux changements rapides dans les applications.

Maintenance complexe : En raison de sa nature basée sur des scripts, UFT peut devenir difficile à maintenir lorsque les applications évoluent rapidement ou lorsque de nouveaux éléments d'interface utilisateur sont introduits.

Flexibilité limitée : La création de scénarios de test complexes ou la gestion de cas de test variés peut être compliquée avec UFT.

Coût élevé : Le coût des licences et de la maintenance d'UFT peut représenter un obstacle pour les entreprises cherchant des solutions plus économiques et agiles pour l'automatisation de leurs tests.

Temps d'exécution élevé : Le temps nécessaire pour exécuter des tests avec UFT peut être important, ce qui peut ralentir le cycle de développement et de déploiement.

Ces limitations ont conduit de nombreuses entreprises à chercher des alternatives plus modernes et flexibles pour leurs besoins en automatisation de tests.



FIGURE 1.2 : logo de l'UFT

1.2.2 Problématique

Dans le contexte actuel de l'automatisation des tests, les équipes de développement sont confrontées à des défis complexes. Alors que des outils traditionnels tels qu'Unified Functional Testing (UFT) ont été largement utilisés par le passé, l'émergence de technologies modernes et d'alternatives telles que Selenium, NUnit et C offre de nouvelles opportunités, mais aussi de nouveaux défis.

L'une des principales limites d'UFT réside dans sa gestion complexe des applications à architecture avancée. Les applications modernes utilisent souvent des architectures complexes, telles que des architectures orientées services (SOA) ou des architectures basées sur des microservices. UFT peut rencontrer des difficultés pour tester efficacement ces applications, ce qui peut entraîner des lacunes dans la couverture des tests et des inefficacités dans le processus de développement.

De plus, UFT peut nécessiter des mises à jour manuelles fréquentes en cas de modifications de l'interface utilisateur. Cela peut entraîner une surcharge de travail pour les équipes de test et une augmentation des coûts de maintenance. De plus, le manque de flexibilité d'UFT face aux technologies émergentes peut limiter sa pertinence dans un environnement en constante évolution, où de nouvelles technologies et frameworks sont régulièrement introduits.

Pour surmonter ces défis, les équipes de développement peuvent envisager d'explorer des alternatives modernes telles que Selenium, NUnit et C. Ces outils offrent souvent une plus grande flexibilité et une meilleure prise en charge des technologies émergentes. De plus, l'adoption de bonnes pratiques d'automatisation, telles que la conception de scripts de test modulaires et maintenables, peut contribuer à améliorer l'efficacité et la maintenabilité des tests automatisés.

En investissant dans la formation et le développement des compétences de leur équipe, les organisations peuvent tirer le meilleur parti des outils d'automatisation des tests disponibles et surmonter les défis associés à l'utilisation d'UFT. En suivant les tendances et en restant à l'affût des développements technologiques, les équipes peuvent également s'assurer que leur approche de l'automatisation des tests reste pertinente et efficace dans un environnement en constante évolution.

Cette problématique soulève des questions cruciales sur l'efficacité et la pertinence des outils

traditionnels d'automatisation des tests dans un environnement logiciel en constante évolution. Comment évoluer vers une approche plus agile et adaptable, capable de répondre aux défis complexes liés aux applications modernes tout en assurant une automatisation des tests efficiente et fiable ? Comment notre framework, basé sur Selenium, NUnit, et C, peut-il représenter une solution innovante pour optimiser le processus de test, améliorer la qualité des tests automatisés, et renforcer la réactivité de notre équipe aux impératifs changeants du développement logiciel ?

Cette problématique guide notre démarche vers l'implémentation d'une solution novatrice, cherchant à dépasser les limites conventionnelles pour répondre aux exigences contemporaines du test logiciel.

1.2.3 Solution Proposée

La solution que nous préconisons repose sur l'implémentation d'un framework d'automatisation des tests, exploitant les technologies modernes telles que Selenium, NUnit et C. L'objectif principal est de résoudre les défis identifiés lors de l'utilisation antérieure d'Unified Functional Testing (UFT) en adoptant une approche plus souple et efficiente pour l'automatisation des tests fonctionnels.

Dans un premier volet, notre solution cible la gestion efficace des applications à architecture complexe. Nous comptons sur Selenium en raison de sa capacité avérée à naviguer dans des interfaces sophistiquées, complétée par l'utilisation de C pour développer des scripts robustes, garantissant une gestion optimale des objets et structures complexes.

Un autre aspect clé de notre solution est la mise en place de mécanismes automatisés de mise à jour en cas de modifications d'interface utilisateur. L'intégration de Selenium permettra la détection automatique de ces changements, facilitant ainsi une mise à jour dynamique des scripts de test. Des stratégies intelligentes seront également utilisées pour minimiser la nécessité d'interventions manuelles fréquentes.

La flexibilité face aux technologies émergentes sera assurée grâce à l'utilisation continue de Selenium, reconnu pour sa polyvalence dans la prise en charge des dernières technologies web. L'intégration de C en tant que langage de programmation offre une adaptabilité accrue aux évolutions technologiques grâce à une communauté active et des mises à jour régulières.

La mise en place d'une stratégie de CI/CD constitue un pilier essentiel de notre solution, visant à automatiser le cycle de vie du logiciel. Cela inclut la configuration d'une pipeline CI/CD pour le build, le test et le déploiement, avec une intégration fluide des tests automatisés pour assurer

la fiabilité du processus.

En parallèle, notre engagement à obtenir la certification ISTQB sur les processus de tests témoigne de notre volonté de renforcer l’expertise et d’appliquer les meilleures pratiques définies par cette norme reconnue. Ainsi, cette solution globale vise à dépasser les limitations de UFT, offrant une approche agile et robuste pour l’automatisation des tests fonctionnels, contribuant ainsi à l’amélioration continue de la qualité des applications testées.

1.2.4 Objectifs

Les objectifs du projet se concentrent sur l’implémentation d’une framework d’automatisation des tests basée sur Selenium, NUnit et C#, ainsi que sur le développement des scripts de test pour une application (IJCORE) . Cette initiative vise spécifiquement à surmonter les limites observées avec l’utilisation antérieure d’Unified Functional Testing (UFT). L’objectif premier est de concevoir une solution robuste qui optimise la gestion des objets complexes et assure la stabilité des scripts de test. Parallèlement, le projet s’engage à configurer une pipeline CI/CD intégrée, automatisant le processus de build, de test et de déploiement, renforçant ainsi la réactivité de l’équipe de développement.

L’adaptabilité aux évolutions technologiques est un axe central, avec l’utilisation de Selenium et C# pour garantir une flexibilité continue face aux technologies émergentes. L’obtention de la certification ISTQB sur les processus de tests est une démarche visant à renforcer l’expertise et à s’aligner sur les meilleures pratiques de l’industrie. Dans l’optique d’optimiser le processus de test, le projet vise à réduire la dépendance aux mises à jour manuelles, favorisant ainsi une maintenance plus efficiente et une maximisation de la couverture des tests. Globalement, les objectifs du projet visent à contribuer à l’amélioration de la qualité de l’application dédiée aux codes-barres des chaises de voiture, en réduisant les risques opérationnels, en renforçant la conformité légale et contractuelle, et en accélérant le processus de test.

1.3 Méthodologie adaptée

Que de suivre un plan rigide, elle permet des ajustements continus en fonction des retours D’expérience et des besoins changeants. Les équipes se réunissent régulièrement pour Discuter des progrès, évaluer les priorités et ajuster le plan d’action en conséquence. Ci-après Un tableau comparatif entre les méthodes agiles et les méthodes classiques/lourdes (RUP,2TUP etc.)

	Méthodes classiques	Méthodes agiles
Planification	Planification détaillée à l'avance	Planification itérative et adaptative
Flexibilité	Moins de flexibilité	Adaptabilité élevée
Livrables	Livraison finale à la fin du projet	Livraison de fonctionnalités à Intervalles réguliers
Transparence	Moins de visibilité sur l'avancement du projet	Transparence élevée grâce aux itérations ,
Délais	Risque de délais plus longs	Livraisons régulières et échéances Plus courtes
Implication du client	Implication limitée jusqu'à la Livraison finale	Implication continue et feedback du client

TABLEAU 1.1 : la comparaison entre les deux méthode agile et classique

D'après les informations présentées précédemment, nous avons opté la méthodologie Scrum dans le cadre de notre projet. Ce choix n'a pas été fait au hasard, car cette méthodologie présente des avantages spécifiques par rapport aux méthodes traditionnelles. Parmi les principaux avantages identifiés qui correspondent à nos objectifs, on peut souligner les suivants :

Premièrement, la flexibilité de cette méthode constitue son principal atout. Scrum a été conçu pour non seulement améliorer la gestion de projet, mais aussi accélérer la réalisation des objectifs. Il est donc essentiel de prévoir une marge de manœuvre pour faire face à l'imprévu, tout en étant capable de refuser des demandes ayant peu d'impact sur le succès du projet en fonction des priorités.

Deuxièmement, Scrum permet d'accroître la productivité en éliminant certaines des "contraintes" associées aux méthodes traditionnelles, telles que la sur documentation ou une formalisation excessive. En outre, Scrum favorise une meilleure qualification de chaque module, ce qui permet d'estimer plus précisément leur charge de travail. Chaque membre de l'équipe peut ainsi se positionner en fonction de la productivité moyenne de l'équipe.

Conclusion

Après avoir présenté la problématique, étudié l'état de l'existant, défini la solution, les objectifs et la méthodologie adoptée, nous introduisons les différents sprints qui suivent pour clarifier notre approche et démontrer la progression réalisée dans la mise en œuvre de notre solution.

ÉTUDE DE DE FAISABILITÉ ET ARCHITECTURE

Plan

Introduction	11
1 Identification des acteurs du système	11
2 Identification des besoins	11
3 Backlog produit	13
4 Architecture globale de la solution	14
Conclusion	18

Introduction

Durant ce chapitre, nous commencerons par décrire les acteurs du système et planifier le backlog du projet. Ensuite, nous détaillerons les besoins fonctionnels et non fonctionnels. Enfin, nous présenterons une architecture générale.

2.1 Identification des acteurs du système

Dans notre solution nous distinguons Trois types d'acteurs avec les rôles suivants :

- **Les Développeurs** : Ce sont les membres de l'équipe chargés de la conception, du développement et de la maintenance du framework d'automatisation de tests.
- **Les Testeurs Fonctionnels** : Ce sont les membres de l'équipe qui sont responsables de la création et de l'exécution des tests fonctionnels.
- **Les Utilisateurs Finaux** : Ce sont les personnes qui utiliseront éventuellement le produit final et dont les besoins doivent être pris en compte lors de la conception du framework.

2.2 Identification des besoins

La qualité de toute application repose largement sur sa conception, ce qui souligne l'importance cruciale de l'analyse des besoins. Cette étape fondamentale consiste à identifier les fonctionnalités requises en examinant attentivement les exigences à la fois fonctionnelles et non fonctionnelles.

2.2.1 Besoins fonctionnels

Le besoin fonctionnel de notre framework d'automatisation des tests est défini par une série d'aspects essentiels pour assurer son efficacité dans un environnement de développement logiciel contemporain. Tout d'abord, le système doit offrir une gestion conviviale des applications complexes, fournissant une interface intuitive permettant aux testeurs de configurer et gérer efficacement des scénarios de test dans des environnements logiciels sophistiqués.

- **Gestion conviviale des applications complexes** : Le système doit offrir une interface intuitive permettant aux testeurs de configurer et gérer efficacement des scénarios de test dans des environnements logiciels sophistiqués. Cela facilite la prise en charge des applications complexes et améliore la productivité des équipes de test.

- **Création de scripts de test robustes** : Un éditeur convivial et un support complet de C# sont nécessaires pour la programmation des tests, garantissant la génération de scripts résilients capables de s'adapter aux évolutions de l'application sans compromettre la fiabilité des tests automatisés.
- **Détection automatique des changements dans l'interface utilisateur** : La dépendance aux mises à jour manuelles est réduite grâce à un mécanisme automatisé intégré au framework, qui alerte en temps réel les testeurs des ajustements nécessaires pour maintenir la stabilité des scripts de test.
- **Flexibilité et adaptabilité aux technologies émergentes** : L'intégration continue de Selenium garantit une compatibilité constante avec les dernières avancées du développement web, tandis que des mécanismes d'extension permettent une adaptation continue aux évolutions futures.
- **Intégration transparente avec la pipeline CI/CD** : Le framework simplifie la configuration de la pipeline CI/CD, permettant le déclenchement automatique des tests aux étapes appropriées du processus de déploiement, assurant ainsi une validation continue du code.
- **Reporting détaillé et visualisation graphique des résultats** : Le besoin fonctionnel comprend la génération automatique de rapports détaillés, offrant une visualisation graphique des résultats et la conservation d'un historique des exécutions pour faciliter l'analyse postérieure.
- **Support multinavigateurs et multiplateformes** : Le framework doit être compatible avec une variété de navigateurs (Chrome, Firefox, Edge, etc.) et fonctionner harmonieusement sur divers systèmes d'exploitation, assurant ainsi sa polyvalence et sa portabilité.

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les paramètres cruciaux qui influent sur la performance globale, la sécurité et l'adaptabilité de notre framework d'automatisation des tests, au-delà des fonctionnalités spécifiques. Tout d'abord, un impératif majeur est d'assurer la performance et la réactivité du framework, avec des temps d'exécution rapides et une capacité à traiter simultanément plusieurs requêtes de tests. La scalabilité est également une priorité, visant à garantir que le framework peut évoluer pour gérer une charge croissante de tests, que ce soit en termes de scénarios, de scripts ou d'utilisateurs simultanés.

- **sécurité** : le framework doit mettre en place des mécanismes robustes de chiffrement des

données et une gestion sécurisée des informations d'authentification pour protéger les données sensibles utilisées dans les scénarios de test. La maintenabilité et l'extensibilité sont également essentielles pour faciliter la maintenance continue du framework et permettre l'ajout aisé de nouvelles fonctionnalités. Cela nécessite une documentation exhaustive, une architecture modulaire et des interfaces bien définies pour l'intégration de plugins.

- **La compatibilité multiplateforme** : constitue un besoin non fonctionnel majeur, garantissant que le framework fonctionne de manière cohérente sur différentes plates-formes et environnements, avec une compatibilité étendue avec les systèmes d'exploitation courants et les versions récentes des navigateurs. Une interface utilisateur conviviale est également une exigence clé pour faciliter la configuration et la gestion des tests, avec une conception intuitive et un support de navigation facile, le tout accompagné d'une documentation accessible.
- **La fiabilité et la tolérance aux erreurs** : sont des critères cruciaux, visant à minimiser les erreurs du framework, fournir des mécanismes de récupération en cas d'échecs et assurer une journalisation détaillée des erreurs. La conformité aux standards de l'industrie est également impérative, exigeant le respect des conventions de codage, la conformité aux normes de sécurité et l'intégration des meilleures pratiques du secteur.
- **la performance en conditions réelles** : est une préoccupation majeure, nécessitant des tests de charge approfondis et une évaluation de la performance dans des environnements de production simulés. En concentrant nos efforts sur ces besoins non fonctionnels, notre framework vise à garantir une expérience utilisateur optimale, une intégration harmonieuse dans l'écosystème du développement logiciel et une conformité rigoureuse aux normes de qualité de l'industrie.

2.3 Backlog produit

Le Backlog du produit constitue un élément central pour notre projet PFE, offrant une vue détaillée des fonctionnalités à développer et des tâches à réaliser. Il se compose d'une liste d'exigences exprimées sous forme d'User Stories, définissant les besoins spécifiques du produit à concevoir. Élaboré en amont du démarrage des sprints, ce Backlog évolue en fonction des exigences du projet, permettant ainsi une adaptation continue aux besoins changeants de développement et assurant l'alignement des efforts de l'équipe sur les objectifs du projet.

ID	Thème	User Story	Priorité	Estimation (jour)
1	Mise en place du Framework d'automatisation	En tant que membre de l'équipe, je veux disposer d'un framework d'automatisation des tests fonctionnels pour faciliter l'exécution et la gestion de nos scénarios de tests.	1	10
2	Développement de scénarios de test	En tant que testeur, je souhaite développer des scénarios de test complets pour valider les fonctionnalités principales de notre application à l'aide du framework d'automatisation.	2	30
3	Configuration de la pipeline CI/CD	En tant que développeur, je souhaite mettre en place une pipeline CI/CD avec Azure DevOps afin d'automatiser le processus de déploiement de notre framework de test et assurer une intégration continue de nos développements.	3	20
3	l'Exécution des Tests sur QC ALM	En tant que développeur, je souhaite mettre en place les artefacts dans les ressources de ALM afin d'exécuter les tests.	4	10

TABLEAU 2.1 : Backlog du produit

2.4 Architecture globale de la solution

Afin de rendre les résultats du projet plus compréhensible, et qui vont être réalisés dans les prochains chapitres, nous avons opté pour l'architecture suivante :

2.4.1 Architecture

La **Figure 2.1** représente l'architecture de notre solution.

Notre framework d'automatisation des tests (GTAF) présente une architecture robuste, optimisée pour la flexibilité, la maintenabilité et l'évolutivité. Voici une vue d'ensemble de ses composantes clés

Notre framework d'automatisation des tests (GTAF) est conçu avec une architecture solide, mettant l'accent sur la flexibilité, la maintenabilité et l'évolutivité. Voici un aperçu détaillé de ses composantes principales :

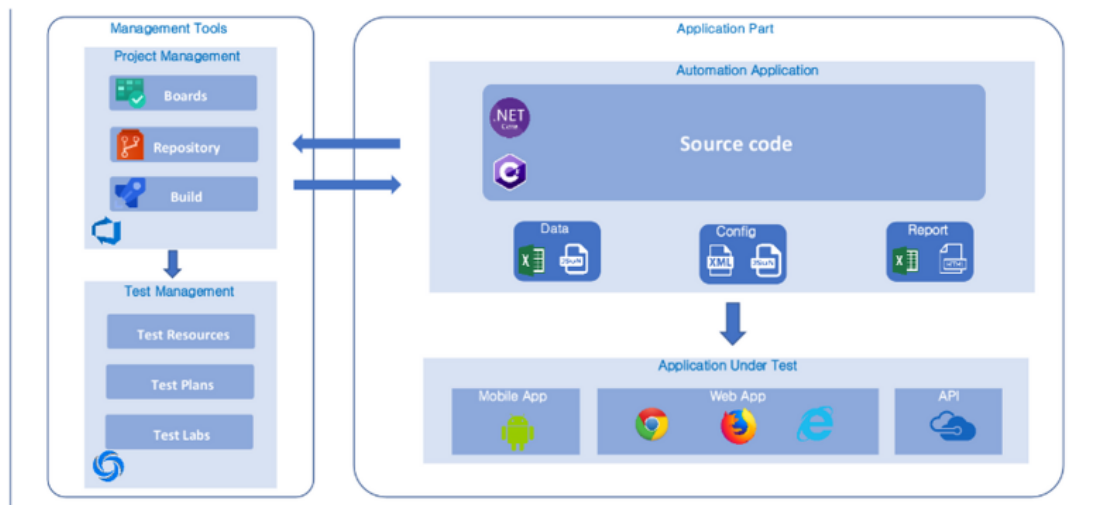


FIGURE 2.1 : Architecture de la solution

Partie Application :

Outils d'Automatisation : Cette section intègre les outils d'automatisation essentiels tels que Selenium, NUnit, et d'autres, qui sont chargés de l'exécution des tests. Ces outils permettent d'interagir avec les différentes couches de l'application et de vérifier son comportement.

Code Source : Les différentes couches du code source, comprenant les modules, classes et interfaces, sont soigneusement organisées pour assurer une compréhension claire et une maintenance efficace. Une structure bien définie facilite également l'ajout de nouvelles fonctionnalités et la correction des bugs.

Application Under Test (AUT) : Qu'il s'agisse d'une application web, mobile ou d'une API, notre framework est conçu pour tester de manière exhaustive l'application sous test. Il peut interagir avec les différents éléments de l'application et valider son fonctionnement conformément aux exigences spécifiées.

Outils de Gestion :

Project Management (Azure DevOps) : Azure DevOps est utilisé pour la gestion globale du projet, y compris la planification, l'estimation et le suivi des sprints. Il permet également de définir et de suivre des indicateurs clés de performance (KPI) liés à l'automatisation des tests, offrant ainsi une visibilité sur l'avancement du projet.

Test Management (ALM) : ALM (Application Lifecycle Management) est maintenu pour gérer l'ensemble du processus de test fonctionnel. Il offre des fonctionnalités telles que la

création de campagnes de tests, la génération de rapports détaillés et l'exécution parallèle des tests. Cela garantit une organisation efficace des tests et une traçabilité complète des résultats.

2.4.2 Les Différentes Couches du framowrk

Dans le domaine du développement logiciel, les tests sont essentiels pour garantir la qualité, la fiabilité et la robustesse des applications. Pour gérer efficacement le processus de test, les équipes adoptent souvent une approche basée sur des couches, où chaque couche a un rôle spécifique à jouer dans l'ensemble du processus de test. Ces couches, souvent appelées "layers" en anglais, comprennent la couche de test, la couche de workflow, la couche d'abstraction et la couche de page.

Chacune de ces couches remplit des fonctions distinctes mais interdépendantes dans le processus global de test logiciel. Dans cet exposé, nous allons examiner en détail chaque couche, explorer son rôle et ses responsabilités, ainsi que les interactions entre elles. En comprenant ces différentes couches, les équipes de développement et de test peuvent mieux organiser et exécuter leurs activités de test, ce qui conduit à des applications de meilleure qualité et à des cycles de développement plus efficaces. La **Figure 2.2** représente Les Différentes Couches du framowrk .

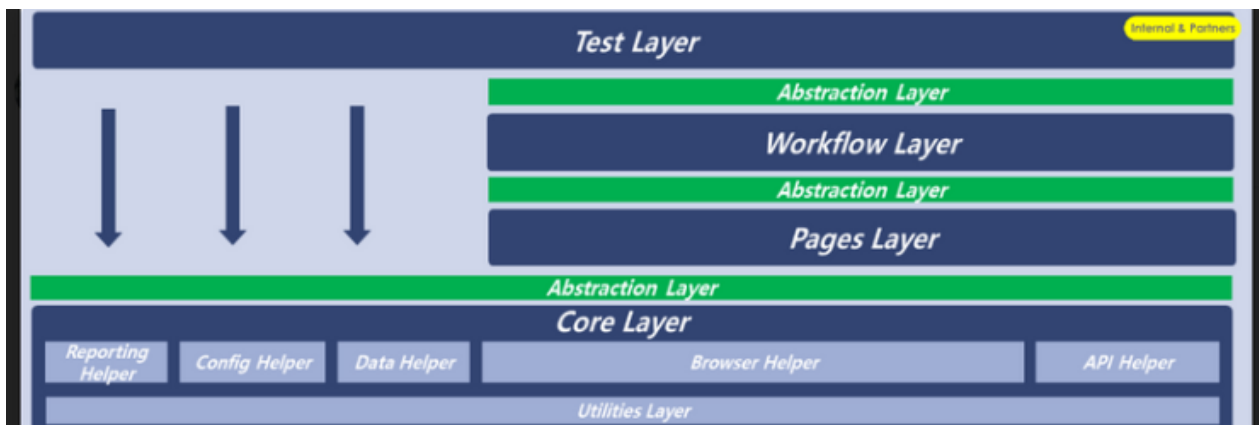


FIGURE 2.2 : Les Différentes Couches du framowrk

Couche de test (Test Layer) :

Cette couche est responsable de l'implémentation de tous les cas de test. Chaque cas de test est représenté par une classe distincte. Les classes de test contiennent le code qui exécute les tests sur l'application sous test (AUT). Elles définissent les actions à effectuer et les vérifications à réaliser pour valider le comportement de l'application à différents niveaux, y compris les tests

unitaires, d'intégration, fonctionnels, et de performance.

Couche de workflow (Workflow Layer) : Dans le cadre du framework GTAF (Keyword-Driven Framework), cette couche contient l'implémentation des mots-clés qui décrivent les actions à effectuer dans les cas de test. Les mots-clés sont des actions ou des opérations de haut niveau qui peuvent être exécutées sur l'application. Ils facilitent la création de tests modulaires et réutilisables en définissant des actions génériques qui peuvent être utilisées dans différents scénarios de test.

Couche d'abstraction (Abstraction Layer) :

Cette couche agit comme une interface entre les tests eux-mêmes et le logiciel sous test. Elle comprend des bibliothèques, des frameworks et d'autres outils qui abstraient les détails techniques du logiciel sous-jacent. Son rôle est de faciliter la création et l'exécution de tests en masquant les complexités techniques sous-jacentes du logiciel. Les interactions dans cette couche se produisent entre les scripts de test et les bibliothèques ou frameworks d'automatisation de test utilisés.

Couche de page (Page Layer) : Dans le contexte des tests d'interface utilisateur (UI), la couche de page concerne la modélisation des pages ou des écrans de l'application et la gestion des interactions utilisateur. Son rôle est de représenter la structure et le comportement des différentes pages ou écrans de l'application, ainsi que les actions que l'utilisateur peut effectuer sur chacun d'eux. Les interactions dans cette couche se produisent entre les tests d'interface utilisateur (souvent des tests automatisés) et les éléments de l'interface utilisateur de l'application testée. Ces différentes couches sont interdépendantes et travaillent ensemble pour garantir la qualité du logiciel à différents niveaux de granularité et d'abstraction.

Couche Core (Core Layer) : Cette couche contient tous les helpers nécessaires à l'exécution des tests automatisés :

Reporting helper : Pour générer des rapports détaillés sur l'exécution des tests, fournissant des informations sur les résultats et les éventuelles erreurs.

Config helper : Pour lire et utiliser les fichiers de configuration contenant des paramètres pour les tests, comme les URL de l'application ou les identifiants de connexion.

Data helper : Pour consommer des données à partir de fichiers Excel, qui peuvent être utilisées dans les tests pour des entrées de données ou des validations.

Browser helper : Pour gérer l'instanciation et la manipulation des navigateurs web lors de

l'exécution des tests.

API helper : Pour gérer les appels d'API vers des services web ou des endpoints, facilitant ainsi les tests d'intégration et la validation des fonctionnalités basées sur les API.

Conclusion

En conclusion, l'architecture globale des tests logiciels, avec ses différentes couches - la couche de test, la couche de workflow, la couche de core, la couche d'abstraction et la couche de page offre une approche organisée et efficace pour garantir la qualité des applications. Chaque couche joue un rôle crucial dans le processus de test, depuis la création et l'exécution des tests jusqu'à la gestion des processus et des interactions avec l'interface utilisateur.

En combinant ces couches de manière cohérente, les équipes de développement peuvent non seulement identifier et corriger les défauts plus rapidement, mais aussi maintenir une documentation claire et une traçabilité des tests. De plus, cette approche favorise une collaboration efficace entre les développeurs, les testeurs et les responsables de la qualité, ce qui contribue à améliorer la productivité et la satisfaction des clients.

En adoptant une approche stratégique des tests logiciels basée sur ces couches, les organisations peuvent élever leur niveau de qualité logicielle tout en optimisant leurs processus de développement, ce qui est essentiel dans un environnement concurrentiel où la rapidité et la fiabilité sont des facteurs clés de succès.

ÉTUDE ET RÉALISATION DU SPRINT 1 :

IMPLÉMENTATION DU FRAMEWORK

Plan

Introduction	20
1 Backlog du sprint	20
2 Choix technologiques	20
3 Réalisation	24
Conclusion	26

Introduction

Le Sprint 1 est dédié à l'implémentation du framework choisi pour le projet. Cette étape initiale est cruciale car elle jette les bases technologiques sur lesquelles reposera le développement ultérieur. L'objectif principal est de sélectionner et d'intégrer le framework approprié, ce qui influence directement la conception, l'architecture et les fonctionnalités de l'application. Ce sprint marque le début de la mise en œuvre concrète du projet, fournissant une base solide pour la progression future du développement.

3.1 Backlog du sprint

Le **Tableau 3.1** présente le Backlog du sprint 1.

ID	Tâche	Estimation (jours)
1	Implémentation initiale du framework	3
2	Configuration de l'environnement de test	3
3	Intégration des bibliothèques et outils	3

TABLEAU 3.1 : Backlog du sprint 1

3.2 Choix technologiques

Le choix de la technologie est un facteur crucial qui influence considérablement les performances de la solution à développer. Ainsi, il est impératif de faire un choix judicieux qui garantisse une flexibilité adaptée aux fonctionnalités spécifiques de la solution. Dans cette section, nous présenterons les diverses approches, techniques et outils sélectionnés pour concrétiser notre solution

3.2.1 Selenium

[2]Selenium est un framework d'automatisation de tests pour les applications web. Son composant central, Selenium WebDriver, agit comme une API permettant d'interagir avec les navigateurs web. Il offre une prise en charge polyvalente des langages de programmation tels que Java, C, Python, Ruby, etc. Selenium simplifie le processus de création et d'exécution de tests automatisés en proposant une approche modulaire. Il permet l'exécution de tests parallèles et la gestion de la distribution des tests sur différentes machines grâce à Selenium Grid. En identifiant précisément les éléments sur les pages web, Selenium facilite la création de tests. Il

offre également une intégration avec des frameworks de tests unitaires comme JUnit ou TestNG et inclut des fonctionnalités telles que des attentes intelligentes. Selenium est compatible avec divers navigateurs et environnements de CI/CD, ce qui en fait un choix essentiel pour garantir la qualité des applications web



FIGURE 3.1 : Selenium

3.2.2 C sharp :

C sharp est un langage de programmation développé par Microsoft, caractérisé par sa simplicité, son modernisme, son orientation objet et sa sécurité. Principalement utilisé pour créer des applications sur la plateforme Microsoft, il englobe le développement d'applications Windows, d'applications web ASP.NET, d'applications mobiles Xamarin, etc. Ce langage de haut niveau est compilé en code intermédiaire (IL) compréhensible par la machine virtuelle .NET (Common Language Runtime, CLR).



FIGURE 3.2 : C sharp

3.2.3 .NET Core

.NET Core est une plateforme de développement créée par Microsoft. Elle est open-source, multiplateforme et permet de créer des applications qui fonctionnent sur Windows, macOS et Linux. .NET Core est idéale pour développer des applications modernes, performantes et évolutives. Elle offre plusieurs caractéristiques principales : elle fonctionne sur différentes

plateformes, est gratuite et améliorée par la communauté de développeurs, et assure des performances élevées grâce à une gestion efficace de la mémoire. De plus, .NET Core est modulaire, permettant d'utiliser seulement les composants nécessaires, et elle est compatible avec Docker, facilitant ainsi le déploiement simplifié des applications.



FIGURE 3.3 : .NET Core

3.2.4 Extent Reports :

Extent Reports est une bibliothèque de génération de rapports pour les tests automatisés en Java. Elle offre des fonctionnalités étendues pour la création de rapports visuels et détaillés sur l'exécution des tests.



FIGURE 3.4 : Extent Reports

3.2.5 JSON.NET :

JSON.NET est une bibliothèque populaire en .NET utilisée pour travailler avec des données au format JSON. Elle offre des fonctionnalités de sérialisation et de désérialisation, permettant la conversion simple entre les objets .NET et le format JSON. JSON.NET prend en charge la personnalisation du processus de sérialisation, la gestion des types complexes, et offre une interface LINQ to JSON pour des opérations plus avancées. Facile à intégrer dans des projets .NET via NuGet,

JSON.NET est apprécié pour sa stabilité, sa performance élevée et sa flexibilité dans le traitement des données JSON



FIGURE 3.5 : JSON.NET

3.2.6 Excel Data Reader :

L'Excel Data Reader est une bibliothèque .NET qui facilite la lecture des données à partir de fichiers Excel (.xls et .xlsx). Cette bibliothèque offre une interface simple pour extraire des données tabulaires à partir de feuilles de calcul Excel sans avoir besoin d'Excel lui-même.



FIGURE 3.6 : Excel Data Reader

3.2.7 HttpClient :

L'**HttpClient** est une bibliothèque .NET utilisée pour effectuer des opérations HTTP dans les applications. Cette classe offre une interface simple et puissante permettant de créer, envoyer et recevoir des requêtes HTTP vers des serveurs distants. Elle fait partie de l'espace de noms **System.Net.Http** et est largement utilisée pour communiquer avec des services web, consommer



FIGURE 3.7 : HttpClient

des API RESTful et effectuer diverses opérations HTTP.

3.2.8 Azure DevOps :

Azure DevOps est une suite d'outils cloud proposée par Microsoft pour la gestion de projets logiciels et le développement d'applications. Elle offre une gamme de fonctionnalités telles que le contrôle de version, la planification de projets, la gestion des exigences, l'intégration continue, le déploiement continu, la gestion des tests et le suivi des bogues,



FIGURE 3.8 : Azure DevOps

3.3 Réalisation

Dans cette section, nous aborderons la configuration nécessaire à notre projet d'implémentation du cœur. Notre objectif est d'installer et de configurer les bibliothèques nécessaires pour le cœur de notre solution en utilisant les packages NuGet appropriés. Cette implémentation se déroulera dans notre environnement de développement, en utilisant le framework dotnet dans sa version 3.1.

3.3.1 Implémentation du CORE

Le CORE d'un framework d'automatisation des tests représente la base fondamentale de son fonctionnement, fournissant les fonctionnalités essentielles pour l'exécution des tests automatisés.

Dans le cas d'un framework basé sur Selenium, cela implique typiquement l'installation de packages NuGet, Selenium WebDriver pour gérer les interactions avec le navigateur. Ces packages comprennent également d'autres fonctionnalités nécessaires comme la génération de rapports ou la gestion des configurations.

Une fois que les packages NuGet sont installés, les fonctionnalités peuvent être accessibles dans le code en important les espaces de noms nécessaires et en utilisant les classes et méthodes fournies. Cela permet de développer des scripts de test automatisés de manière efficace et fonctionnelle.

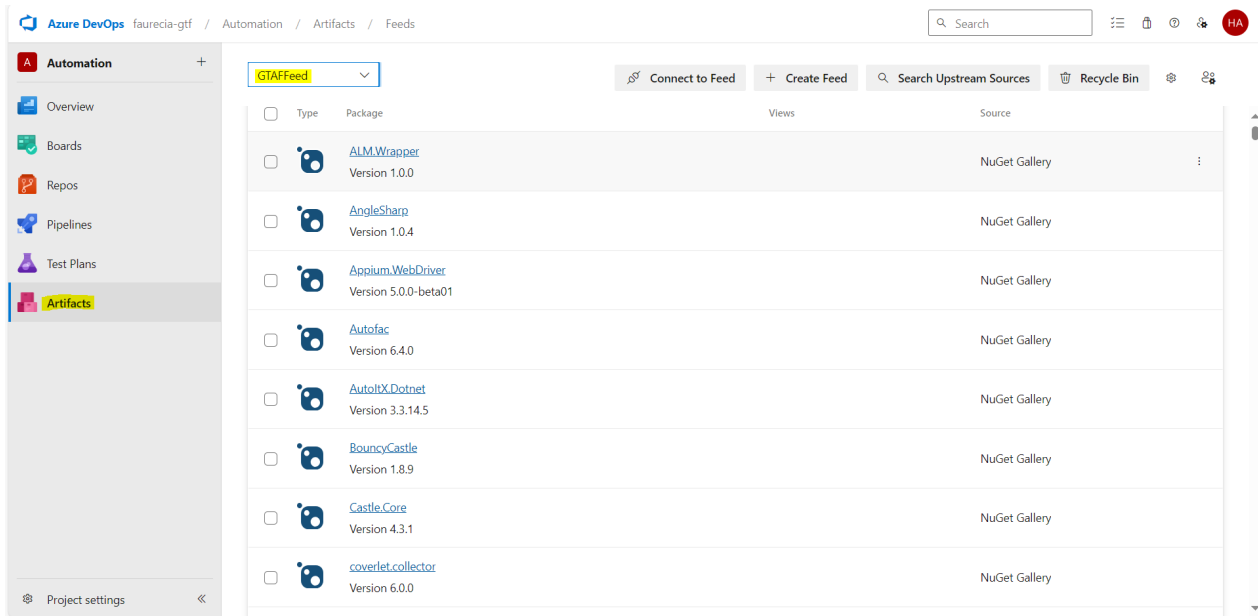


FIGURE 3.9 : Exploration des Packages NuGet dans Azure DevOps pour une Intégration Fluide dans Visual Studio Code

3.3.2 Configuration des Paramètres du Gestionnaire de Packages dans Visual Studio

Les configurations supplémentaires sont nécessaires dans les paramètres du gestionnaire de packages, comme la spécification de sources personnalisées ou la gestion des versions. Ces ajustements peuvent être effectués depuis l'environnement de développement intégré ou d'autres outils spécifiques au gestionnaire de packages.

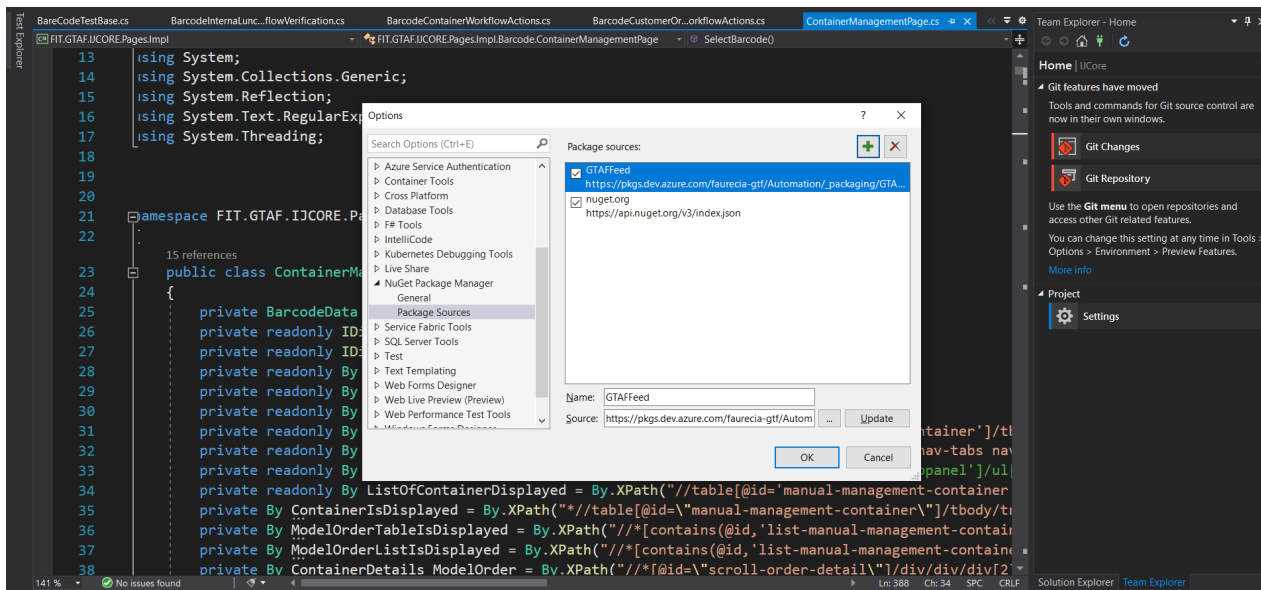


FIGURE 3.10 : Configuration des Paramètres du Gestionnaire de Packages

Conclusion

Dans ce chapitre, nous avons suivi le déroulement du premier sprint de notre projet. Nous avons d'abord examiné les choix technologiques et détaillé l'architecture de l'environnement, puis nous avons présenté la mise en place de notre solution à travers des captures d'écran illustratives. Le prochain chapitre nous permettra de poursuivre cette exploration en décrivant le déroulement du deuxième sprint, offrant ainsi une continuité dans le développement et l'évolution de notre projet.

ÉTUDE ET RÉALISATION DU SPRINT 2 :

DÉVELOPPEMENT DES SCÉNARIOS DE

TESTS

Plan

Introduction	28
1 Backlog du sprint	28
2 Choix technologiques	29
3 Présentation du processus de tests	29
4 Réalisation	30
Conclusion	37

Introduction

Dans cette partie consacrée au développement des scénarios de tests, nous explorerons en détail le processus d'écriture et de création des cas de test pour notre application. Les scénarios de tests jouent un rôle crucial dans l'assurance de la qualité des logiciels en garantissant que toutes les fonctionnalités sont testées de manière exhaustive et rigoureuse.

Au cours de cette étape, nous allons aborder les différentes méthodes et approches utilisées pour concevoir des scénarios de tests complets et efficaces, en nous basant sur les spécifications fonctionnelles de l'application et les besoins des utilisateurs. Nous examinerons également comment ces scénarios sont intégrés dans notre framework d'automatisation des tests, assurant ainsi une exécution automatisée et répétable des tests.

En explorant cette partie, nous aurons un aperçu approfondi de la manière dont les scénarios de tests sont conçus et exécutés, et comment ils contribuent à garantir la fiabilité et la robustesse de notre application dans des environnements variés.

4.1 Backlog du sprint

Nous présenterons maintenant le quatrième chapitre dans lequel nous mettons le point sur le Développement des scénarios de test . Nous commencerons donc par rédiger le Backlog du sprint, le choix des technologies utilisé ainsi que l'architecture de la mise du système, et nous terminerons par la partie réalisation. Ce **Tableau** présente le Backlog du sprint 2.

ID	Tâche	Estimation (jours)
1	Collaborer avec l'équipe fonctionnelle pour comprendre le processus des tests	9
2	Développement des scripts de test automatisés	17

TABLEAU 4.1 : Backlog du sprint 2

4.2 Choix technologiques

4.2.1 NUnit :

NUnit, un Framework de test unitaire, facilite la création et l'exécution des tests grâce à des annotations simples pour définir les cas de test et les assertions. Il permet également une organisation méthodique des tests en regroupant les suites logiques, simplifiant ainsi leur exécution sélective. L'outil offre une gamme d'assertions prédéfinies pour évaluer les résultats, assurant la conformité des comportements attendus du code.

Il gère également les configurations et les nettoyages avant/après chaque test, assurant ainsi un état initial cohérent. Avec une intégration facile dans Visual Studio, NUnit offre une expérience de développement fluide pour les développeurs travaillant dans cet environnement. De plus, sa capacité à exécuter des tests en parallèle accélère le processus de test, en faisant un outil puissant pour assurer la qualité du code en automatisant les tests, les rendant plus accessibles, faciles à maintenir et intégrés dans le flux de développement



FIGURE 4.1 : Nuint

4.3 Présentation du processus de tests

Dans la phase de développement des scénarios de test, nous nous engageons dans un processus itératif et méthodique pour créer des cas de test complets et efficaces. Tout d'abord, nous nous appuyons sur les exigences fonctionnelles et non fonctionnelles du système pour définir les cas de test appropriés. Ces cas de test sont conçus pour couvrir tous les aspects importants du système, en tenant compte des différents chemins d'exécution possibles et des scénarios d'utilisation variés.

Nous nous appuyons également sur le modèle en V, qui associe chaque niveau de développement à un niveau de test correspondant. Cela signifie que nous élaborons des tests de composants pour évaluer les parties individuelles du système, des tests d'intégration pour vérifier les interactions entre ces composants, des tests système pour évaluer le système dans son ensemble, et enfin des tests

d'acceptation pour valider que le système répond aux besoins et attentes des utilisateurs finaux.

Pendant le développement de ces scénarios de test, nous accordons une attention particulière à la qualité et à la couverture des tests. Nous nous efforçons de garantir que chaque fonctionnalité critique est testée de manière approfondie et que tous les cas de test sont pertinents et réalistes. Nous collaborons étroitement avec les parties prenantes, notamment les développeurs, les responsables produits et les utilisateurs finaux, pour nous assurer que les scénarios de test répondent aux exigences et aux attentes de tous.

En fin de compte, ce processus de développement des scénarios de test vise à garantir que le système est stable, fiable et conforme aux normes de qualité établies. Il contribue à identifier les défauts potentiels dès les premières phases du développement, ce qui permet de les corriger rapidement et de réduire les risques associés au projet.

Cette Figure illustre Phase de définition des tests dans le cycle de développement en V.

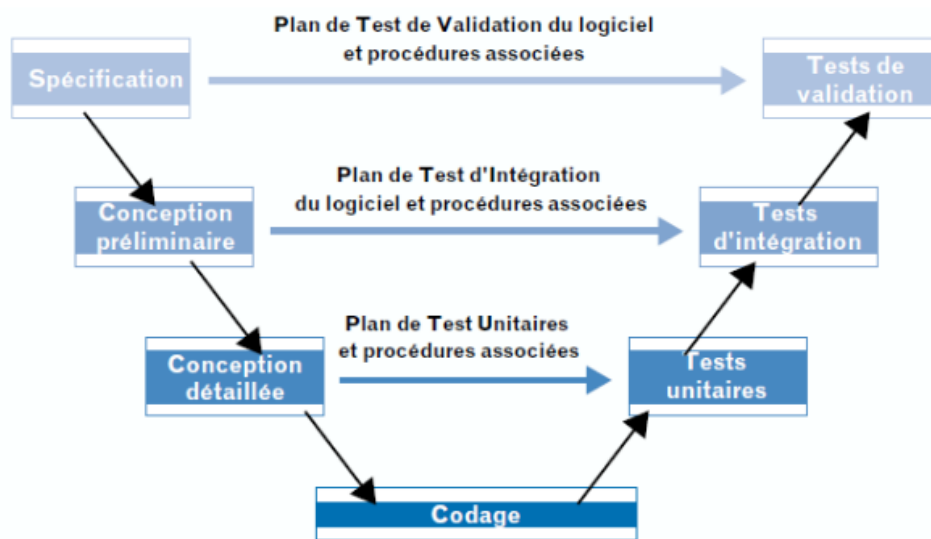


FIGURE 4.2 : Phase de définition des tests

4.4 Réalisation

La mise en œuvre de tests automatisés efficaces nécessite une approche méthodique et rigoureuse, depuis l'identification des besoins métier jusqu'à la définition du workflow des tests. Dans cette section, nous détaillerons les étapes clés que nous avons suivies pour développer et mettre en place ces

scénarios de test automatisés. En commençant par l'analyse des besoins métier, nous avons travaillé en étroite collaboration avec les parties prenantes pour comprendre les fonctionnalités requises et les cas d'utilisation associés. Ensuite, nous avons regroupé les scénarios de test existants et évalué leur pertinence par rapport aux nouveaux besoins. Pour les fonctionnalités non couvertes, nous avons développé de nouveaux scénarios de test, en nous appuyant sur une collaboration étroite avec l'équipe fonctionnelle pour garantir une compréhension approfondie des processus métier. En parallèle, nous avons géré les données d'entrée nécessaires à l'exécution des tests et défini le workflow détaillé de chaque scénario. Cette approche méthodique a permis de créer des tests automatisés fiables et cohérents, prêts à être intégrés dans notre processus de développement logiciel.

4.4.1 Identification des besoins métier :

Pour répondre de manière exhaustive aux exigences métier fournies par le groupe métier, nous adoptons une approche méthodique. Tout d'abord, nous organisons des réunions avec les parties prenantes, y compris les représentants du groupe métier et les intervenants clés, pour discuter et clarifier les besoins et les attentes. Ces réunions servent également à valider les exigences et à s'assurer que toutes les parties impliquées ont une compréhension commune des objectifs du projet. Cette figure représente la capture d'écran de l'application utilisée pour effectuer les tests.

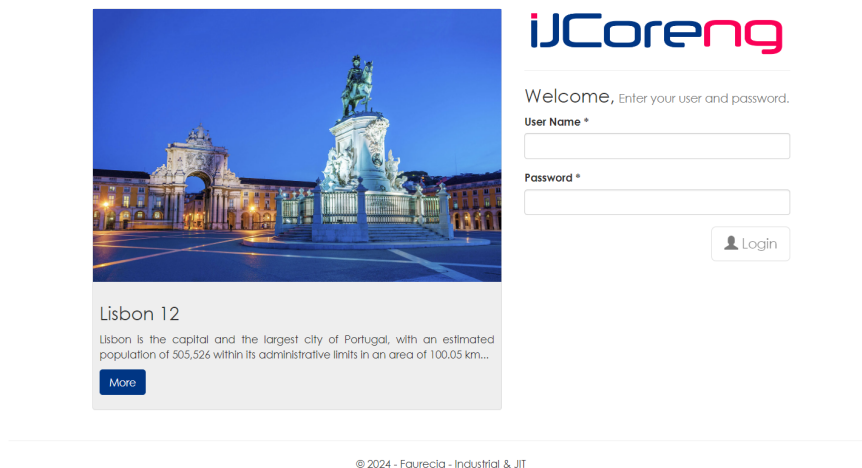


FIGURE 4.3 : Capture d'écran de l'application testée

Une fois les besoins clarifiés et validés, nous documentons de manière exhaustive les fonctionnalités requises. Cela inclut la description détaillée de chaque fonctionnalité, ses objectifs, ses critères de succès et les conditions d'utilisation. De plus, nous identifions et documentons les cas d'utilisation

associés à chaque fonctionnalité, décrivant les différentes interactions des utilisateurs avec le système et les résultats attendus dans divers scénarios.

En documentant de manière détaillée les exigences métier et les cas d'utilisation, nous nous assurons que toutes les parties prenantes ont une référence claire et précise des besoins du projet. Cela facilite la communication, réduit les risques d'interprétation incorrecte des exigences et contribue à la livraison d'un produit final qui répond aux attentes et aux besoins des utilisateurs

4.4.2 Regroupement des scénarios de test :

Pour garantir une mise en œuvre efficace des exigences métier, nous suivons un processus rigoureux. Tout d'abord, nous passons en revue les exigences métier fournies par le groupe métier, en analysant chaque aspect pour en comprendre la portée et les implications. Ensuite, nous procédons à un regroupement des fonctionnalités connexes en ensembles logiques, en identifiant les liens et les dépendances entre elles. Cette approche nous permet de mieux organiser le développement et de prioriser les efforts en fonction des besoins les plus critiques.

Parallèlement, nous examinons les scénarios de test manuels existants qui sont pertinents pour chaque ensemble de fonctionnalités identifié. Cela nous donne une base solide pour commencer à élaborer nos plans de test automatisés. Nous évaluons ensuite ces scénarios de test en fonction de leur importance pour les tests automatisés, en tenant compte de facteurs tels que la fréquence d'utilisation de la fonctionnalité, son impact sur l'expérience utilisateur, et sa criticité pour le bon fonctionnement du système.

En priorisant les scénarios de test de cette manière, nous nous assurons de concentrer nos ressources sur les aspects les plus critiques du système, tout en maximisant l'efficacité de nos tests automatisés. Cela nous permet de garantir la qualité du produit final tout en respectant les délais et les contraintes de développement.

Cette figure représente un exemple de test élaboré par les équipes fonctionnelles, décrivant les actions à effectuer et les vérifications à réaliser.

	Step Name	Description	Expected Result
	Step 1	Connect to IJCore	Home page displayed
	Step 2	Access to customer order to get barcodes datas	Customer order management page displayed successfully
	Step 3	Set search filters Loop overall number of calloff and save all realted data (jit order, delivery order, module, barcode) Access to container management and save all containers ID	List of calloff containing barcode data should be created successfully
	Step 4	For each saved barcode, build the expected barcode value digit by digit Verify generated barcode	Generated barcode by the automation should have the same value as the barcode generated in IJCore

FIGURE 4.4 : un exemple de scénario de test

4.4.3 Évaluation des scénarios existants :

Pour garantir une compréhension claire et précise des besoins du projet, nous organisons des réunions régulières avec les parties prenantes clés. Ces réunions nous permettent de discuter en profondeur des exigences métier, de clarifier tout point ambigu et de valider les attentes pour assurer l'alignement de tous les acteurs impliqués.

En parallèle, nous passons en revue les scénarios de test existants pour identifier ceux qui peuvent être réutilisés avec des ajustements mineurs ou sans modification. Cette approche nous permet de capitaliser sur les efforts déjà investis dans le développement des scénarios de test, ce qui contribue à une utilisation efficace des ressources et à une accélération du processus de test.

Une fois les scénarios de test réutilisables identifiés, nous documentons soigneusement nos conclusions et recommandations. Cette documentation comprend une description détaillée de chaque scénario de test, ainsi que des indications sur les ajustements éventuels à apporter. Ces informations sont précieuses pour les futures phases de développement et de test, car elles fournissent une base solide pour la planification et l'exécution des tests.

4.4.4 Développement de nouveaux scénarios :

Pour répondre aux besoins non couverts par les scénarios de test existants, nous entamons la conception de nouveaux scénarios de test automatisés. Pour ce faire, nous collaborons étroitement

avec l'équipe fonctionnelle afin de comprendre en détail les processus métier associés à chaque nouveau scénario identifié.

Cette collaboration nous permet de saisir pleinement les exigences et les spécifications de chaque processus métier, ce qui est essentiel pour garantir une couverture adéquate des fonctionnalités dans nos tests automatisés. Nous nous appuyons sur cette compréhension approfondie pour rédiger des scripts de test détaillés, en suivant les meilleures pratiques de l'automatisation des tests.

Lors de la rédaction des scripts de test, nous nous efforçons de capturer tous les cas de test pertinents, en nous assurant de couvrir tous les scénarios d'utilisation possibles. Nous prenons également en compte les différents chemins d'exécution, les entrées utilisateur variées et les résultats attendus pour chaque scénario.

En appliquant les meilleures pratiques de l'automatisation des tests, tels que l'utilisation de structures de données modulaires, la gestion des dépendances et la gestion des erreurs, nous nous assurons que nos scripts de test sont robustes, maintenables et évolutifs.

4.4.5 Gestion des données d'entrée :

Pour assurer l'exécution efficace des tests, il est essentiel de collecter et d'organiser les données d'entrée nécessaires de manière appropriée. Nous adoptons une approche méthodique pour cette tâche cruciale.

Tout d'abord, nous identifions les données d'entrée requises pour chaque scénario de test. Cela peut inclure des informations telles que les identifiants d'utilisateur, les paramètres de configuration, les données de test spécifiques, etc.

Ensuite, nous stockons ces données dans un fichier de données dédié, tel qu'une feuille de calcul Excel. L'utilisation d'un tel fichier nous permet de centraliser et d'organiser les données de manière structurée. De plus, il offre la flexibilité nécessaire pour que les données soient facilement accessibles et modifiables. Cette approche garantit une gestion efficace des données et facilite les mises à jour ultérieures si nécessaire.

Parallèlement, nous accordons une attention particulière à la qualité et à la validité des données collectées. Nous effectuons des vérifications rigoureuses pour nous assurer que les données sont complètes, précises et cohérentes. Cela garantit des tests fiables et reproductibles, en réduisant les risques d'erreurs dues à des données incorrectes ou incomplètes.

Cette Figure représente un exemple de fichier de données dans lequel chaque entrée est

correctement organisée et formatée.

	A	B	C	D	E	F	G	H	I	J
1	User	Password	NumberOfCalloff	FeatureIdentification	BarcodeScreen	BarcodeType	Module	Plant	CheckAllGroupeFeaturesForFeaturePartPresence	OriginDefinitionForCustomer
2	GTFAuto	UCOREAuto1	1	QCCC	CustomerOrder	BZD_Barcode	IRS	FALSE		Customer_Origin
3										
4										
5										
6										
7										
8										
9										
10										

FIGURE 4.5 : un exemple de fichier de données

4.4.6 Définition du workflow des tests :

Pour chaque scénario de test, nous adoptons une approche systématique en le décomposant en étapes distinctes. Dans cette démarche, nous identifions les actions spécifiques à effectuer ainsi que les vérifications à réaliser à chaque étape du processus. Cette décomposition nous permet d'assurer une couverture complète du test et de garantir que toutes les interactions avec l'application sont prises en compte.

Une fois les étapes définies, nous créons un document détaillé décrivant le flux de chaque test. Ce document fournit une vue d'ensemble claire du déroulement du test, en incluant toutes les actions à effectuer et les vérifications à réaliser. Nous nous assurons que chaque interaction potentielle avec l'application est couverte, garantissant ainsi une validation complète du comportement de l'application.

Avant de passer à l'étape d'exécution des tests, nous testons et validons le workflow des tests pour assurer sa cohérence et son efficacité. Nous passons en revue chaque étape du processus, en nous assurant que les actions et les vérifications sont correctement alignées avec les objectifs du test. Cette étape préliminaire nous permet de détecter et de corriger tout problème ou incohérence dans le workflow, garantissant ainsi que les tests seront exécutés de manière fluide et sans heurts. Cette Figure représente un exemple de fichier du workflow dans lequel chaque entrée est correctement organisée et formatée.

1	StepNumber	Action	Order	Execute	V12	V14
2	1	LoginWithCorrectCredential	1	TRUE	TRUE	TRUE
3	1	VerifyHomePageDisplayed	2	TRUE	TRUE	TRUE
4	2	AccessToCustomerOrder	3	TRUE	TRUE	TRUE
5	2	VerifyCustomerOrderMangementScreenDisplayed	4	TRUE	TRUE	TRUE
6	3	SetSearchFiltersForCustomerManagementOrderN	5	TRUE	TRUE	TRUE
7	3	VerifyListOfCallOffDisplayedNewWorkflow	6	TRUE	TRUE	TRUE
8	3	SaveCallOffBaiscData	7	TRUE	TRUE	TRUE
9	4	GetExpectedBarcodeValueForCustomerOrder	8	TRUE	TRUE	TRUE
10	4	VerifyBarcodeGeneratedValueForCustomerOrder	9	TRUE	TRUE	TRUE
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						
21						
22						
23						

FIGURE 4.6 : un exemple de fichier workflow

La construction de fichiers de données et de workflows efficaces nécessite en effet un développement de tests organisé autour de différentes couches. Ces couches comprennent généralement :

Couche Data : Cette couche concerne la gestion des données utilisées dans les tests. Elle implique la création de fichiers ou de structures de données qui contiennent les informations nécessaires aux tests, telles que les entrées utilisateur, les valeurs attendues, etc.

Couche Page : La couche de page est responsable de l'interaction avec l'interface utilisateur de l'application. Elle fournit des méthodes et des fonctions pour accéder aux éléments de l'interface utilisateur, ce qui permet d'abstraire la logique de test de la manipulation directe de l'interface utilisateur.

Couche Test : Cette couche représente les scénarios de test proprement dits. Elle utilise les données de la couche Data et les méthodes de la couche Page pour exécuter les actions nécessaires et vérifier les résultats obtenus.

Workflow : Le workflow définit la séquence des actions à effectuer lors de l'exécution des tests. Il orchestre les différentes étapes du processus de test, en spécifiant l'ordre dans lequel les tests doivent être exécutés et en gérant les transitions entre les différentes étapes.

En utilisant .NET pour le développement de tests, ces différentes couches peuvent être implémentées à l'aide de frameworks et de bibliothèques spécifiques à .NET, tels que Selenium pour l'automatisation des tests Web ou NUnit pour le framework de test unitaire. Dans la partie

implémentation du framework, chaque couche est généralement expliquée en détail, mettant en lumière son rôle et sa contribution à la construction des tests automatisés.

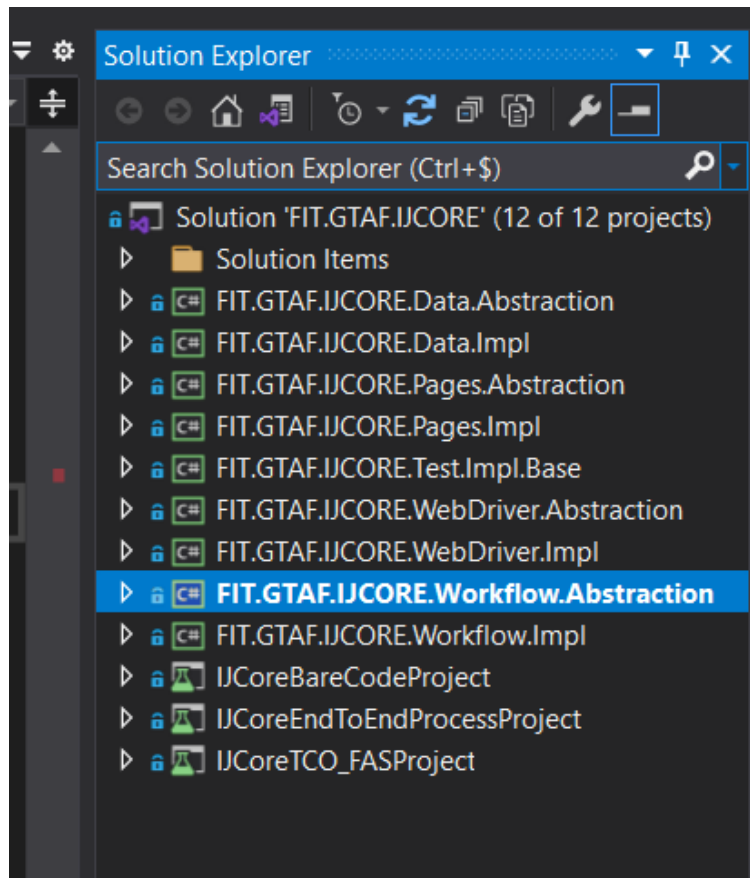


FIGURE 4.7 : les différentes couches utiliser pour exécuter le test

Conclusion

Dans ce chapitre, nous avons détaillé le déroulement du deuxième sprint de notre projet. Nous avons entamé cette phase en décrivant les choix technologiques spécifiques qui orientent notre travail et soutiennent nos objectifs. En outre, nous avons examiné le processus de tests, mettant en lumière les étapes clés de cette phase cruciale du développement logiciel. Enfin, nous avons illustré la mise en place de notre solution à travers des captures d'écran, offrant ainsi une vision concrète de notre progression. Le prochain chapitre sera axé sur la mise en place d'un pipeline CI/CD avec Azure DevOps.

ÉTUDE ET RÉALISATION DU SPRINT

3 : MISE EN PLACE D'UN PIPELINE CI/CD DANS AZURE DEVOPS

Plan

Introduction	39
1 Backlog du sprint	39
2 Choix technologiques	39
3 Architecture de l'environnement	41
4 Réalisation	42
Conclusion	48

Introduction

Dans ce chapitre, nous plongeons dans l'un des aspects cruciaux de notre projet : la mise en place d'un pipeline CI/CD dans Azure DevOps. Cette étape est essentielle pour assurer la qualité et la stabilité de notre solution logicielle tout au long de son cycle de développement. À travers cette introduction, nous allons explorer les motivations derrière l'adoption de cette approche, les bénéfices qu'elle apporte à notre processus de développement, ainsi que les défis auxquels nous pourrions être confrontés lors de sa mise en œuvre.

5.1 Backlog du sprint

Le **Tableau 5.1** présente le backlog du sprint 3.

ID	User Story	Technical Story	Estimation (jours)
3	En tant que développeur je veux préparer un pipeline CI/CD avec AzureDevops	Vérifier la connexion et les autorisations du projet	1
		Créer le fichier de configuration YAML pour Configurer les étapes de build	7
		Configuration pour le déploiement des artefacts	7
		Tests du Pipeline	3

TABLEAU 5.1 : Backlog du Sprint 3

5.2 Choix technologiques

5.2.1 Azure DevOps :

Azure DevOps est une suite d'outils qui facilite le développement logiciel en fournissant des services qui couvrent tout le cycle de vie d'une application, de la planification et du suivi du travail à la création, au test et au déploiement, en passant par la gestion du code source.

Après avoir défini Azure DevOps et ses fonctionnalités, nous allons maintenant comparer la création d'une pipeline de CI/CD dans deux outils différents : Azure DevOps et Jenkins. Voici un tableau comparatif.

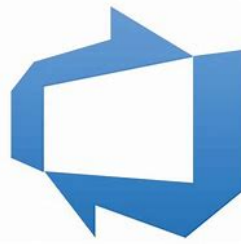


FIGURE 5.1 : Azure DevOps

Étape	Azure DevOps	Jenkins
Configuration du projet	Créer un projet dans Azure DevOps et définir le référentiel Git.	Installer et configurer Jenkins sur un serveur ou un environnement.
Configuration du pipeline de CI	Utiliser l'interface graphique d'Azure DevOps pour créer un pipeline basé sur YAML ou YAML multi-étapes.	Configurer le pipeline à l'aide d'un fichier Jenkinsfile dans le référentiel Git ou via l'interface utilisateur de Jenkins.
Définition des étapes de build	Définir les étapes de build, telles que la compilation, les tests unitaires, l'analyse statique du code, etc., dans le fichier YAML.	Configurer les étapes de build dans le Jenkinsfile, en spécifiant les commandes de build, les outils à utiliser, etc.
Intégration avec les outils de test	Intégrer les outils de test tiers tels que NUnit, MSTest, ou xUnit pour exécuter les tests unitaires	Installer et configurer les plugins de test appropriés dans Jenkins pour exécuter les tests unitaires.
Déploiement des artefacts	Utiliser les tâches de déploiement intégrées dans Azure DevOps pour déployer les artefacts générés sur des environnements cibles, tels que le déploiement sur Azure App Service, Azure Kubernetes Service	Configurer les étapes de déploiement dans le Jenkinsfile pour déployer les artefacts sur les serveurs cibles à l'aide d'outils tels que SSH, Docker, ou Kubernetes.
Gestion des notifications et des rapports	Configurer des notifications par e-mail, des alertes et des rapports de build à l'aide des fonctionnalités intégrées d'Azure DevOps.	Installer et configurer des plugins Jenkins pour envoyer des notifications par e-mail, générer des rapports de build et gérer les alertes.

TABLEAU 5.2 : Comparaison entre Azure DevOps et Jenkins

⇒ Dans le choix entre Azure DevOps et Jenkins pour la mise en place de pipelines CI/CD, il est important de considérer la collaboration étroite avec Microsoft Visual Studio, ce qui rend

Azure DevOps incontournable pour notre société. Cette collaboration permet une intégration native avec Visual Studio, offrant ainsi une expérience fluide pour les développeurs habitués à cet environnement. En ayant la possibilité de configurer et de gérer les pipelines CI/CD directement depuis Visual Studio, le processus est simplifié, accélérant ainsi l'adoption des pratiques de CI/CD au sein de notre équipe de développement

5.3 Architecture de l'environnement

Pour l'architecture de notre solution avec Azure DevOps[5], nous pouvons envisager une approche modulaire qui prend en compte les différents aspects du développement logiciel, de la planification à la surveillance en passant par le développement, les tests et le déploiement. Voici une proposition d'architecture :

Azure Boards : Cette composante servira de point central pour la gestion du projet, permettant la planification des fonctionnalités, la création de tâches et de sprints, ainsi que le suivi de l'avancement du projet.

Azure Repos : Repositories pour le stockage et la gestion du code source. Chaque modification du code déclenchera un processus CI/CD via Azure Pipelines.

Azure Pipelines : C'est ici que les pipelines CI/CD seront configurés pour automatiser le processus de build, de test et de déploiement. Les étapes du pipeline peuvent être définies en utilisant la configuration YAML, offrant une flexibilité et une reproductibilité maximales.

Azure Artifacts : Repositories pour le stockage des artefacts générés lors des processus de build. Cela peut inclure des packages, des binaires, des images de conteneurs, etc.

Azure Test Plans : Utilisé pour la planification, l'exécution et le suivi des tests. Il permet également la collaboration entre les membres de l'équipe pour la création et l'exécution de scénarios de test manuels.

Azure Monitor : Outil de surveillance pour suivre les performances des applications déployées, détecter les problèmes et collecter des données de télémétrie pour l'analyse ultérieure.

Azure DevOps API : Une API RESTful qui permet d'automatiser et d'intégrer des fonctionnalités Azure DevOps dans d'autres outils et processus.

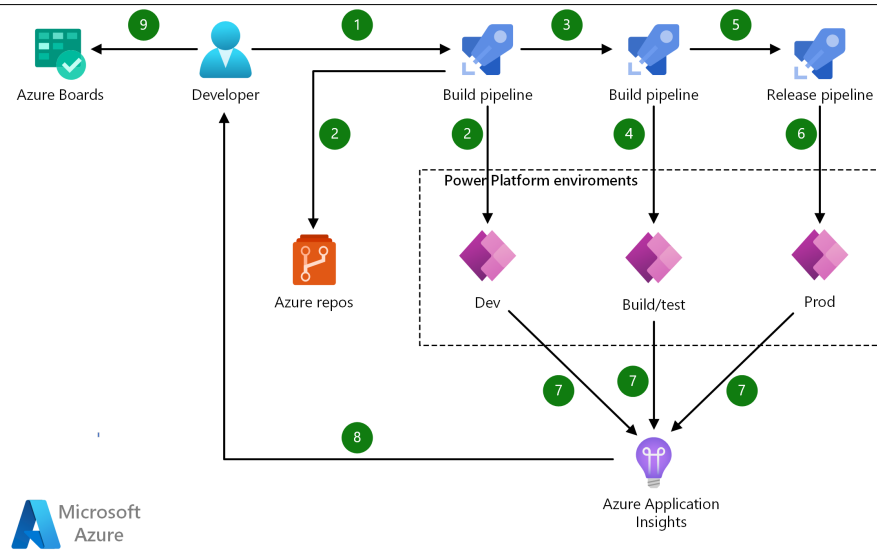


FIGURE 5.2 : L'architecture de notre solution avec Azure DevOps

Cette architecture offre une approche complète pour la gestion du cycle de vie des applications, de la planification à la surveillance en passant par le développement et le déploiement. Elle exploite pleinement les fonctionnalités et les services offerts par Azure DevOps pour fournir une solution robuste et intégrée pour notre projet.

5.4 Réalisation

Ce sprint est dédié à la mise en place d'une pipeline CI/CD avec Azure DevOps pour notre projet. Cette étape a été une étape cruciale dans notre démarche de développement logiciel. Cette pipeline automatisée nous permet de compiler, tester et déployer nos applications de manière efficace et cohérente. Dans ce processus, plusieurs étapes clés sont impliquées, notamment la construction (build), le nettoyage (clean) de la solution, et le déploiement des artefacts. De plus, nous avons intégré nos fichiers de workflow et de données, ainsi que nos projets sous forme de points zip, pour assurer une livraison fluide de notre logiciel. Dans cette section, nous détaillerons chaque étape de notre réalisation, en mettant en lumière les décisions prises et les configurations effectuées pour garantir le bon fonctionnement de notre pipeline CI/CD.

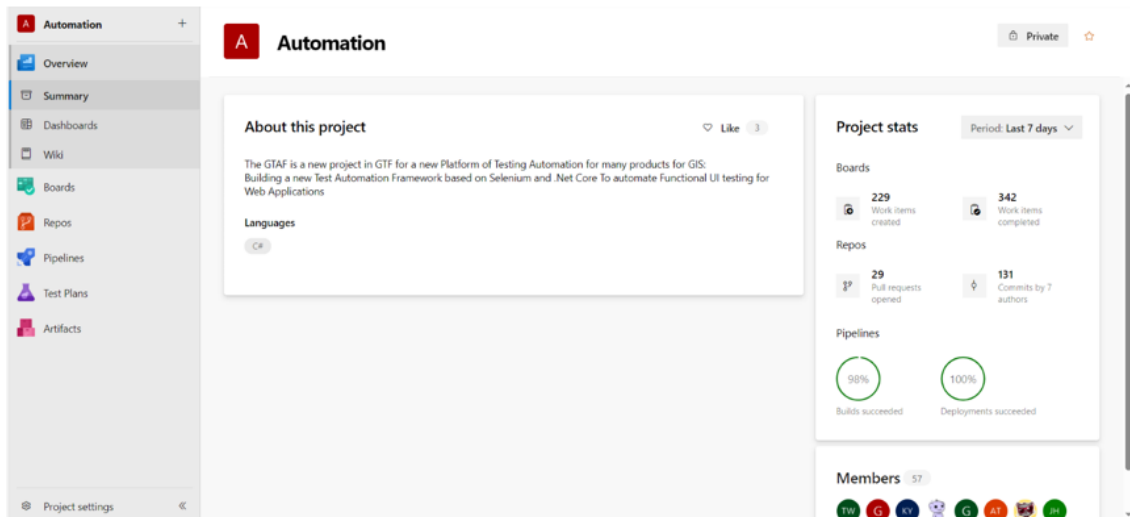


FIGURE 5.3 : Interface de Azure DevOps

Nous avons présenté le processus détaillé pour la mise en place d'une pipeline CI/CD comme le montre la **Figure 5.4**.

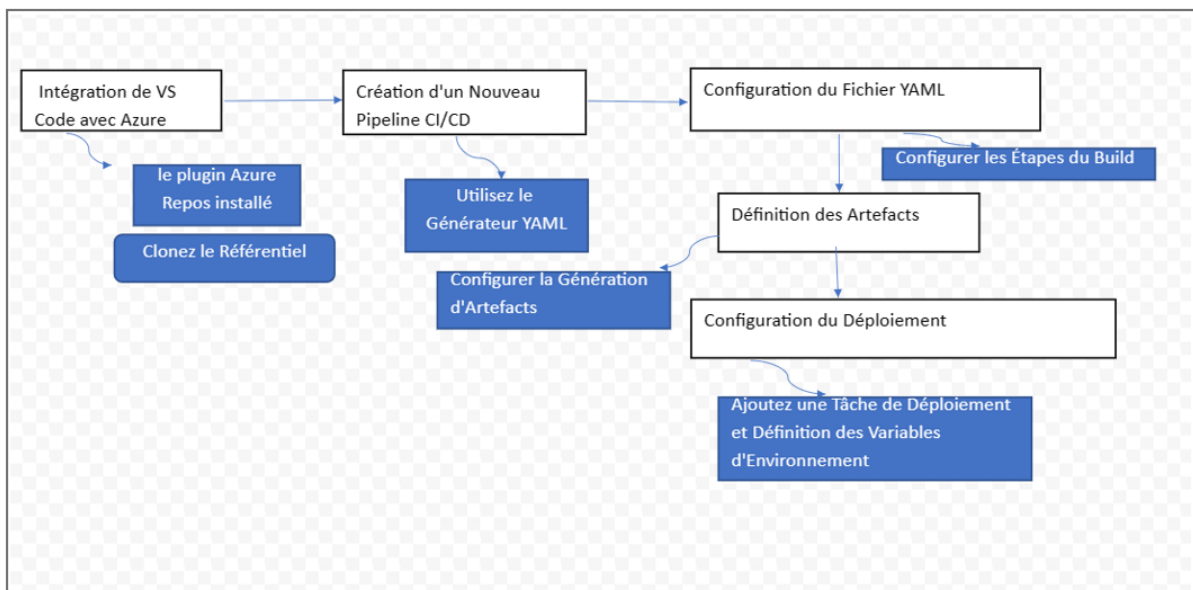


FIGURE 5.4 : le processus pour la mise en place d'une pipeline CI/CD

5.4.1 Intégration de Visual Studio Code avec Azure Repos

L'intégration de Visual Studio Code avec Azure Repos constitue une étape fondamentale dans le processus de développement logiciel. Cette intégration permet aux développeurs de travailler

efficacement sur leur code source directement depuis l'interface familière de VS Code. Une fois le plugin Azure Repos installé, les développeurs peuvent cloner le référentiel Azure Repos de leur projet et commencer à collaborer sur le code. Cette étape facilite la gestion du code source, la collaboration entre les membres de l'équipe et la synchronisation des modifications avec Azure Repos, contribuant ainsi à un flux de travail plus fluide et plus efficace.

5.4.2 Création d'une nouvelle pipeline CI/CD

La création d'une nouvelle pipeline CI/CD dans Azure DevOps est une étape essentielle pour automatiser le processus de développement, de test et de déploiement de notre application. Cette pipeline nous permet de définir les étapes spécifiques à exécuter à chaque modification du code source, garantissant ainsi une livraison continue et fiable de notre logiciel. À travers l'interface intuitive d'Azure DevOps, nous pouvons configurer les différentes phases de la pipeline, telles que la compilation, les tests unitaires, les tests d'intégration, et le déploiement sur les environnements de développement, de test et de production. La création de cette pipeline constitue une étape cruciale dans l'adoption des pratiques DevOps, favorisant une meilleure collaboration entre nos équipes de développement et d'opérations, et assurant une mise en production rapide et sécurisée de notre application.

5.4.3 Configuration des Étapes de Build dans Azure DevOps

Pour configurer les étapes de build dans Azure DevOps en utilisant un fichier YAML, nous devons accéder aux paramètres de notre pipeline dans notre projet Azure DevOps. Une fois dans les paramètres, nous pouvons éditer le fichier YAML de notre pipeline. Dans ce fichier YAML, nous pouvons définir les différentes étapes de build en utilisant la syntaxe YAML appropriée. Cela inclut la spécification des tâches à exécuter, telles que la compilation du code source, l'exécution de tests unitaires, la vérification de la qualité du code, etc. Nous pouvons également définir des dépendances entre les différentes étapes, des variables d'environnement et d'autres paramètres nécessaires à l'exécution de notre build. Une fois que nous avons configuré les étapes de build dans notre fichier YAML, nous devons nous assurer de valider le fichier pour détecter les erreurs de syntaxe ou de configuration. Ensuite, nous enregistrons nos modifications

pour appliquer la configuration à notre pipeline. Enfin, nous pouvons déclencher l'exécution du pipeline pour compiler, tester et déployer notre application conformément à la configuration spécifiée dans le fichier YAML

5.4.4 Configuration des Artefacts dans la Pipeline CI/CD Azure DevOps

Pour spécifier et configurer les artefacts dans notre pipeline CI/CD Azure DevOps, nous ajoutons une étape de publication à notre fichier YAML. Cette étape de publication informe Azure DevOps des fichiers ou dossiers générés par le build qui doivent être enregistrés en tant qu'artefacts pour une utilisation ultérieure dans les étapes de déploiement. Nous définissons les chemins des fichiers ou dossiers à publier ainsi que leur destination dans le pipeline. Ensuite, nous avons des tâches d'archivage qui regroupent ces fichiers ou dossiers dans des archives compressées, telles que des fichiers zip, pour faciliter leur stockage et leur transport. Enfin, nous utilisons des tâches de publication pour téléverser ces archives vers Azure DevOps, où elles sont disponibles en tant qu'artefacts de build pour les étapes suivantes de la pipeline. Cette approche garantit que tous les artefacts nécessaires à notre application sont correctement générés, archivés et publiés, prêts à être utilisés dans le processus de déploiement.

Dans le cadre de notre projet, nous avons identifié trois principaux artefacts :

L'application exécutable : il s'agit du fichier principal résultant du build, contenant le code de l'application prêt à être déployé.

Les packages de données : ces fichiers contiennent les données nécessaires au bon fonctionnement de l'application, telles que les configurations spécifiques et les ressources.

Les packages de workflow : ils comprennent les scripts ou les configurations de flux de travail nécessaires pour l'exécution de l'application.

Nous spécifions ces artefacts dans notre pipeline en utilisant la syntaxe YAML appropriée, en précisant les chemins des fichiers ou dossiers à inclure et en définissant leur destination dans le pipeline. Une fois cette étape complétée, nos artefacts seront prêts à être utilisés dans les étapes de déploiement ultérieures pour assurer une livraison continue et efficace de notre application.

Le **Figure 5.5** et **Figure 5.6** représentent les différentes étapes à suivre de la pipeline

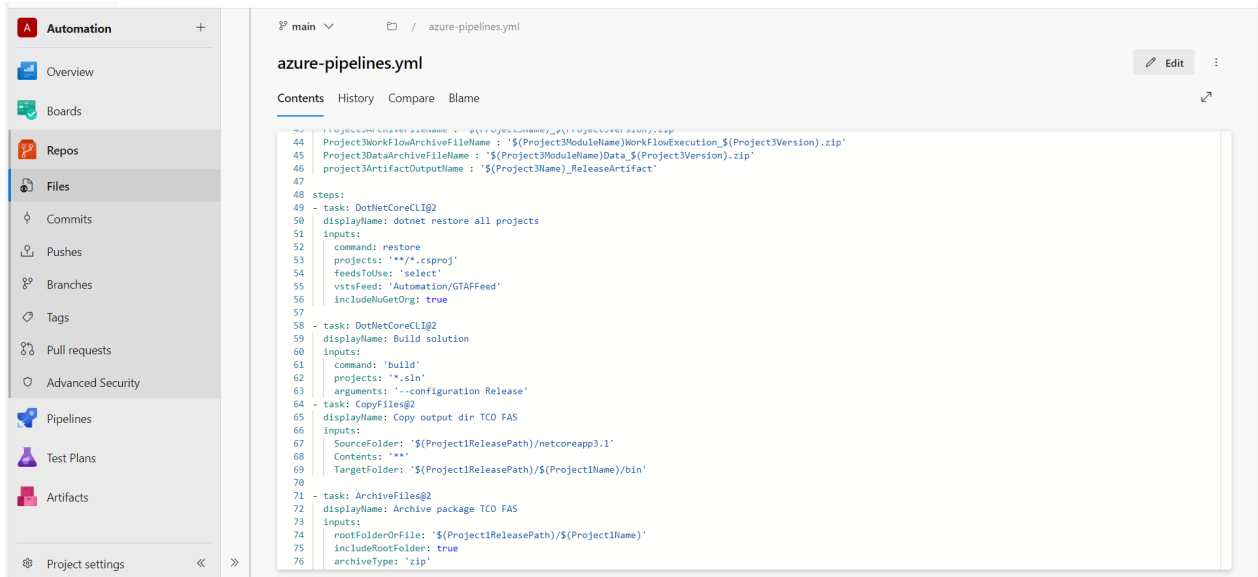


FIGURE 5.5 : Étape de la pipeline de build

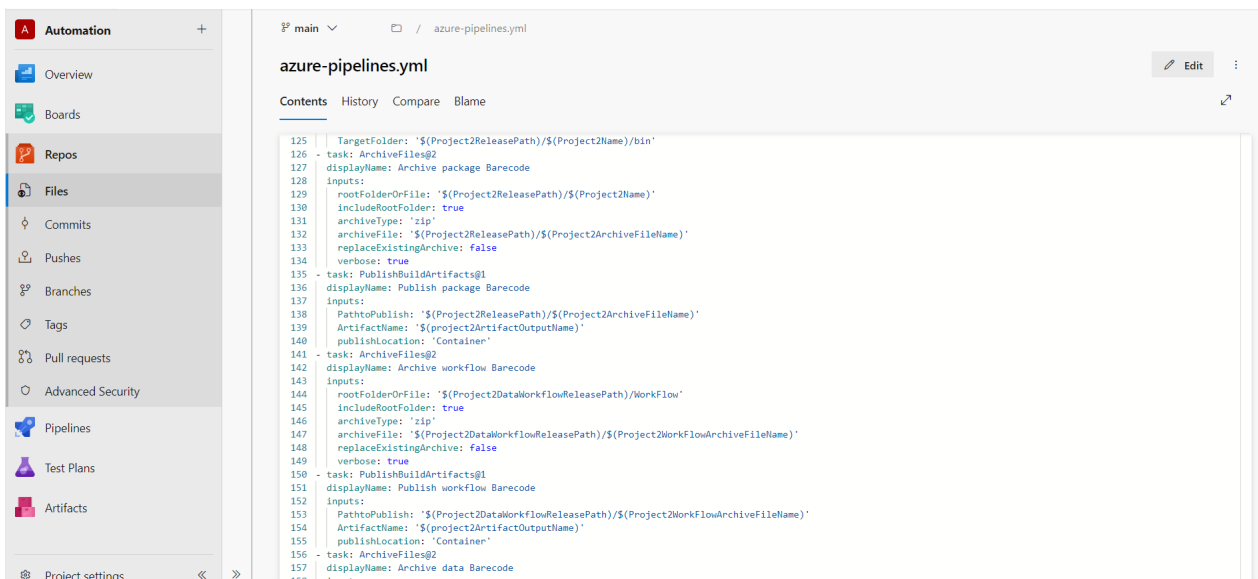


FIGURE 5.6 : les étapes de copie, d'archivage et de publication

les différentes étapes à suivre de la pipeline :

Construction de la solution : Cette étape consiste à compiler la solution à l'aide de DotNetCoreCLI, en spécifiant le mode de configuration Release. Cela garantit que votre code est compilé dans un environnement optimisé pour la production.

Copie des fichiers de sortie : Après la compilation, cette tâche copie les fichiers de sortie générés (tels que les fichiers binaires, les fichiers de configuration, etc.) vers un répertoire spécifié. Cela prépare les fichiers pour les étapes suivantes du processus ou pour un déploiement ultérieur.

Archivage du package : Cette étape consiste à regrouper les fichiers de sortie dans un format zip. L'archivage facilite le stockage et le transport des fichiers, en les combinant en un seul fichier compressé.

Publication du package : Après l'archivage, cette tâche publie l'archive zip en tant qu'artefact dans un référentiel ou un système de gestion des versions. Cela permet aux membres de l'équipe d'accéder facilement aux fichiers compilés pour une utilisation ultérieure ou pour le déploiement.

Archivage du workflow : De manière similaire à l'étape d'archivage du package, cette étape archive les fichiers de workflow dans un format zip. Cela regroupe et compresse les fichiers de workflow pour un stockage et un transport efficaces.

Publication du workflow : Après l'archivage, cette tâche publie l'archive zip des fichiers de workflow en tant qu'artefact. Cela rend les fichiers de workflow accessibles à toute l'équipe, facilitant ainsi la collaboration et le déploiement.

Archivage des données : Cette étape est chargée d'archiver les données pertinentes dans un format zip. Cela peut inclure des données de configuration, des fichiers de configuration, des scripts SQL, ou tout autre type de données nécessaires au fonctionnement de votre application.

Publication des données : Enfin, après l'archivage, cette tâche publie l'archive zip des données en tant qu'artefact. Cela garantit que les données sont disponibles pour toute l'équipe, ce qui peut être crucial pour la configuration et le déploiement de l'application.

La **Figure 5.8** présente le résultat de la génération des artefacts.

Chapitre 5. Étude et réalisation du sprint 3 :Mise en place d'un pipeline CI/CD dans Azure DevOps

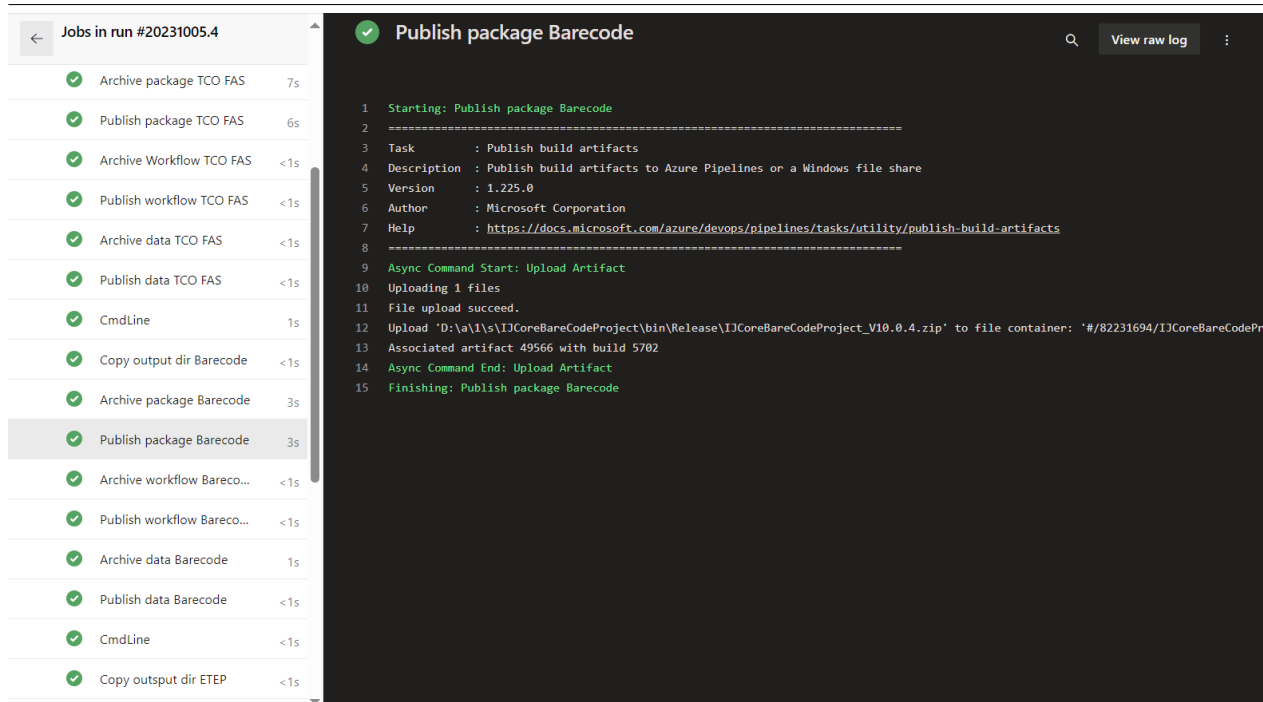


FIGURE 5.7 : La figure représente l'exécution de la pipeline CI/CD dans Azure DevOps,

Artifacts	
Published	
Name	Size
> ALMDeliveryScript	3 KB
▼ IJCoreBarcodeProject_ReleaseArtifact	23 MB
BarcodeData_V10.0.4.zip	598 KB
BarcodeWorkFlowExecution_V10.0.4.zip	435 KB
IJCoreBarcodeProject_V10.0.4.zip	22 MB
> IJCoreEndToEndProcessProject_ReleaseArtifact	24 MB
> IJCoreTCO_FASProject_ReleaseArtifact	44 MB

FIGURE 5.8 : les artefacts générés

Conclusion

Dans ce cinquième chapitre, nous avons décrit le déroulement du troisième sprint. Nous avons commencé par présenter les technologies utilisées ainsi que l'architecture de l'environnement. Ensuite, nous avons expliqué en détail la mise en place d'un pipeline CI/CD avec Azure DevOps, en utilisant des captures d'écran pour illustrer les différentes étapes. Le prochain

chapitre sera consacré à présenter en détail le déroulement du dernier sprint.

ÉTUDE ET RÉALISATION DU SPRINT 4 :

L'EXÉCUTION DES TESTS AVEC QC

ALM

Plan

Introduction	51
1 Backlog du sprint	51
2 Choix technologiques	51
3 Architecture de l'environnement	53
4 Réalisation	54
Conclusion	58

Introduction

Après avoir terminé la configuration du pipeline pour la génération des artefacts (exécutable, packages de workflow et de données), nous entamons la phase de déploiement de ces artefacts sur ALM et l'exécution des tests.

6.1 Backlog du sprint

Le **Tableau 6.1** présente le Backlog du sprint 4.

ID	User Story	Technical Story	Estimation (jours)
4	En tant qu'ingénieur qualité, je veux exécuter un test.	Mise en place des artefacts dans ALM	1
		Génération d'un script de test de type VAPI XP	2
		Ajout de la configuration adéquate	4
		Exécution du test	5

TABLEAU 6.1 : Backlog du sprint 4

6.2 Choix technologiques

6.2.1 Application Lifecycle Management(ALM)

[6]Le Application Lifecycle Management (ALM) est un processus intégré de gestion du cycle de vie des logiciels, qui englobe toutes les phases de développement, de déploiement et de maintenance d'une application logicielle

Dans la section Tests de QC ALM, nous avons :

- Ressources de test
- Plan de test
- Laboratoire de test



FIGURE 6.1 : logo de l'ALM

Ressources de test (Test resources) Les ressources de test font référence aux divers éléments nécessaires pour réaliser les tests. Cela peut inclure des documents, des environnements de test, des scripts, des jeux de données, et des outils spécifiques. Dans QC ALM, les ressources de test sont gérées pour assurer que toutes les composantes nécessaires aux tests sont disponibles, correctement configurées, et accessibles par l'équipe de test.

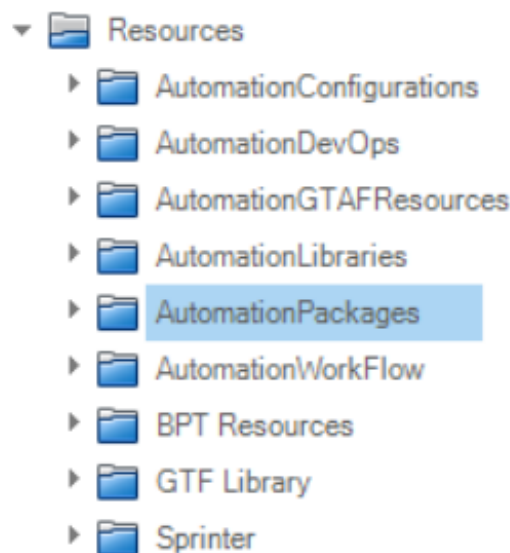


FIGURE 6.2 : Ressources de test

Plan de test (Test plan) Le plan de test est un document ou une section dans QC ALM où les tests sont définis et organisés. Il comprend des informations sur les objectifs des tests, la portée, les critères de réussite, les méthodes de test, les cas de test individuels, et les exigences associées. Le plan de test sert de feuille de route pour l'équipe de test, détaillant ce qui doit être testé, comment les tests doivent être exécutés, et quels sont les résultats attendus.

Laboratoire de test (Test lab) Le laboratoire de test est la section de QC ALM où les tests sont exécutés. Ici, les cas de test définis dans le plan de test sont rassemblés en ensembles

de tests et exécutés dans un environnement contrôlé. Le laboratoire de test permet de suivre l'exécution des tests, de consigner les résultats, et de gérer les défauts ou bugs détectés. C'est l'endroit où l'on surveille et contrôle le processus de test en temps réel.

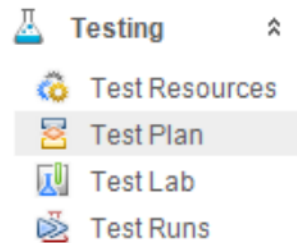


FIGURE 6.3 : la section des Tests

6.3 Architecture de l'environnement

Cette **Figure** illustre l'architecture de Workflow pour Déployer les Artéfacts Après leur Génération

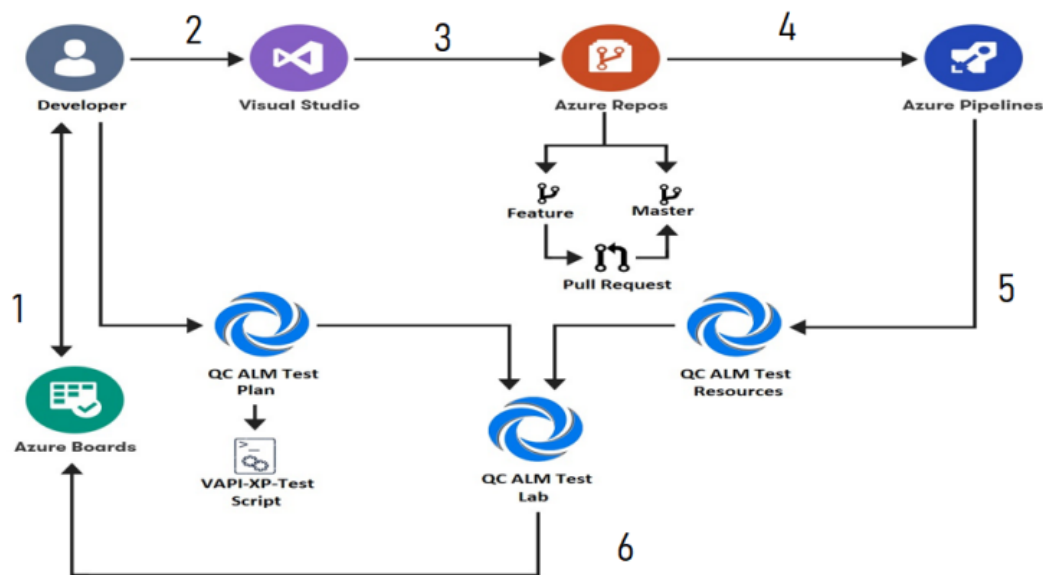


FIGURE 6.4 : l'architecture de Workflow

Dans le cadre de notre processus de développement et de déploiement, nous avons mis en place une architecture de workflow efficace pour le déploiement des artéfacts générés. Ce workflow est essentiellement composé des étapes suivantes :

1 : Consultation du Plan de Test ALM et Mise à Jour des Statuts sur Azure Boards : L'ingénieur en automatisation se réfère au plan de test ALM pour identifier les étapes

de conception des cas de test. Une fois les cas de test identifiés, les statuts correspondants sont mis à jour sur les tableaux Azure Boards pour refléter l'avancement du travail.

2 : Développement des Scénarios des Cas de Test : L'ingénieur en automatisation rédige et développe les scénarios des cas de test conformément aux spécifications définies dans le plan de test ALM. Ces scénarios décrivent les étapes à suivre pour tester les fonctionnalités du logiciel.

3 : Fusion des Mises à Jour : L'équipe de développement intègre les modifications dans la branche principale du code source et effectue des revues de code pour assurer la qualité des modifications. Cette étape garantit que les changements apportés sont cohérents avec les objectifs du projet et respectent les normes de qualité.

4 : Déclenchement du Pipeline CI : À chaque fusion dans la branche principale, le pipeline CI est automatiquement déclenché. Ce pipeline compile le code, exécute les tests et génère les artefacts de build. Cela permet de vérifier rapidement l'intégrité du code et de détecter les erreurs potentielles.

5 : Déploiement des Artefacts sur les Ressources de Test ALM : Les artefacts générés sont déployés sur les environnements de test définis dans ALM à l'aide d'outils de déploiement automatisés. Cela permet de simuler les conditions réelles dans lesquelles le logiciel sera utilisé, afin de détecter et de résoudre les problèmes avant la mise en production.

6 : Exécution des Tests dans le Laboratoire de Test ALM : Les cas de test automatisés sont exécutés sur les ressources de test déployées. Les résultats sont collectés et analysés pour identifier les échecs et les succès. Cette étape permet de valider le bon fonctionnement du logiciel et de garantir sa qualité avant sa mise en production.

6.4 Réalisation

6.4.1 Implémentation du Déploiement d'Artefacts dans ALM et Exécution des Tests avec Rapports d'Analyse

Dans cette section, nous présenterons le travail accompli tout au long de ce sprint pour parvenir à la mise en œuvre du déploiement des artefacts. Nous avons commencé par déployer les artefacts, comme le montre la **Figure 6.5** suivante.

puis nous avons configuré un script de type VAPI XP Test au niveau du cas de test dans le

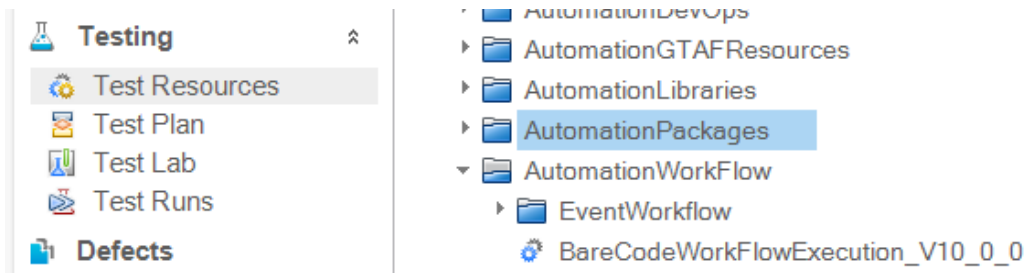


FIGURE 6.5 : Artefacts(packages et workflow)

plan de test pour établir la connexion entre les étapes et les données requises comme le montre la **Figure 6.6** .

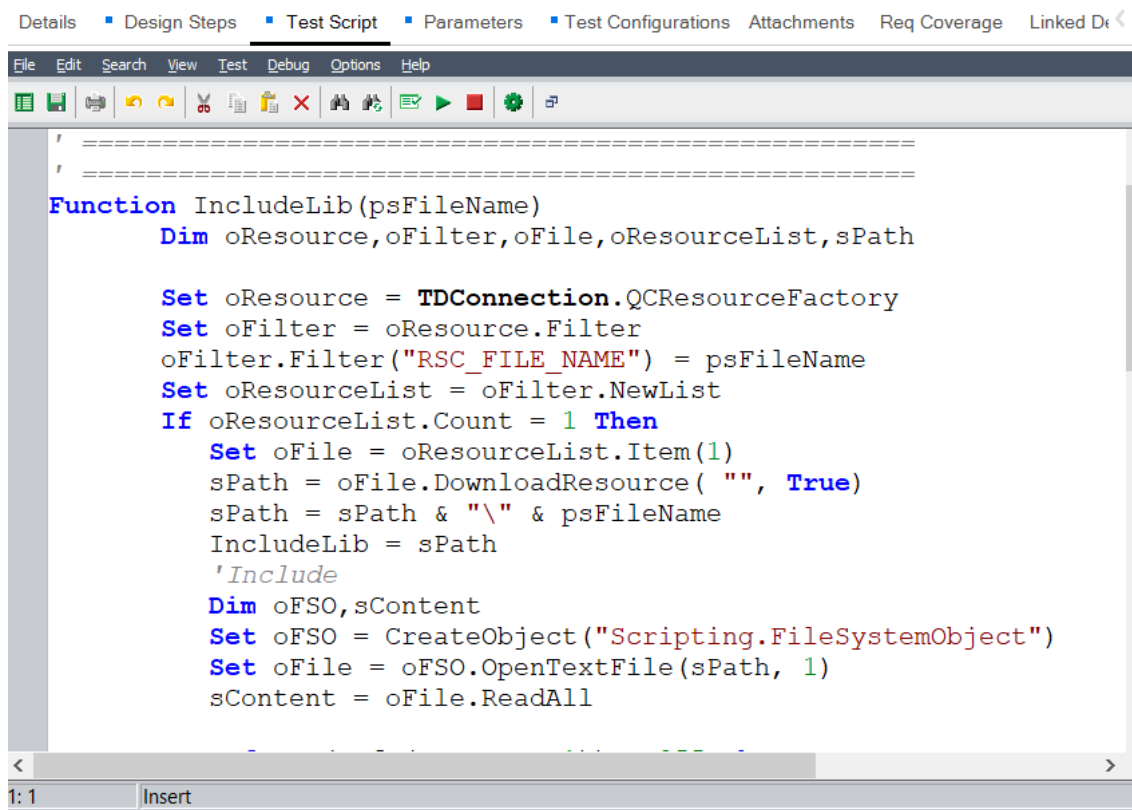
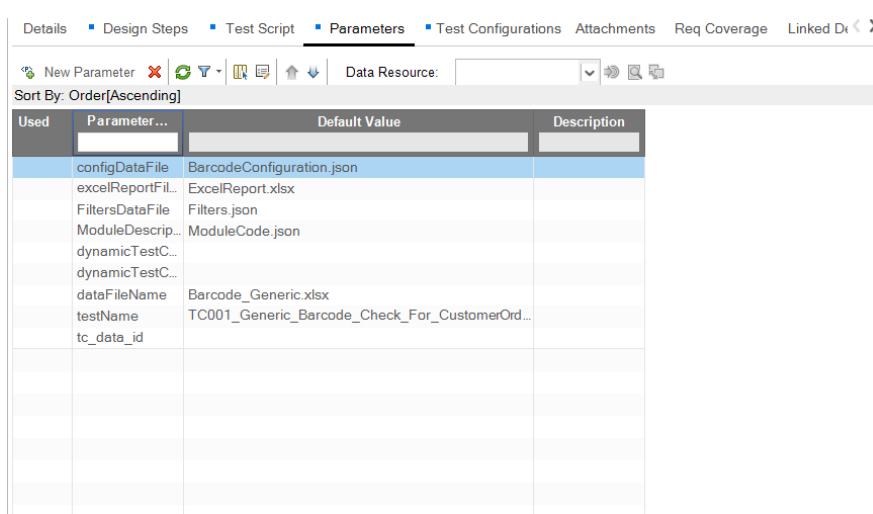


FIGURE 6.6 : La configuration du script de type VAPI XP

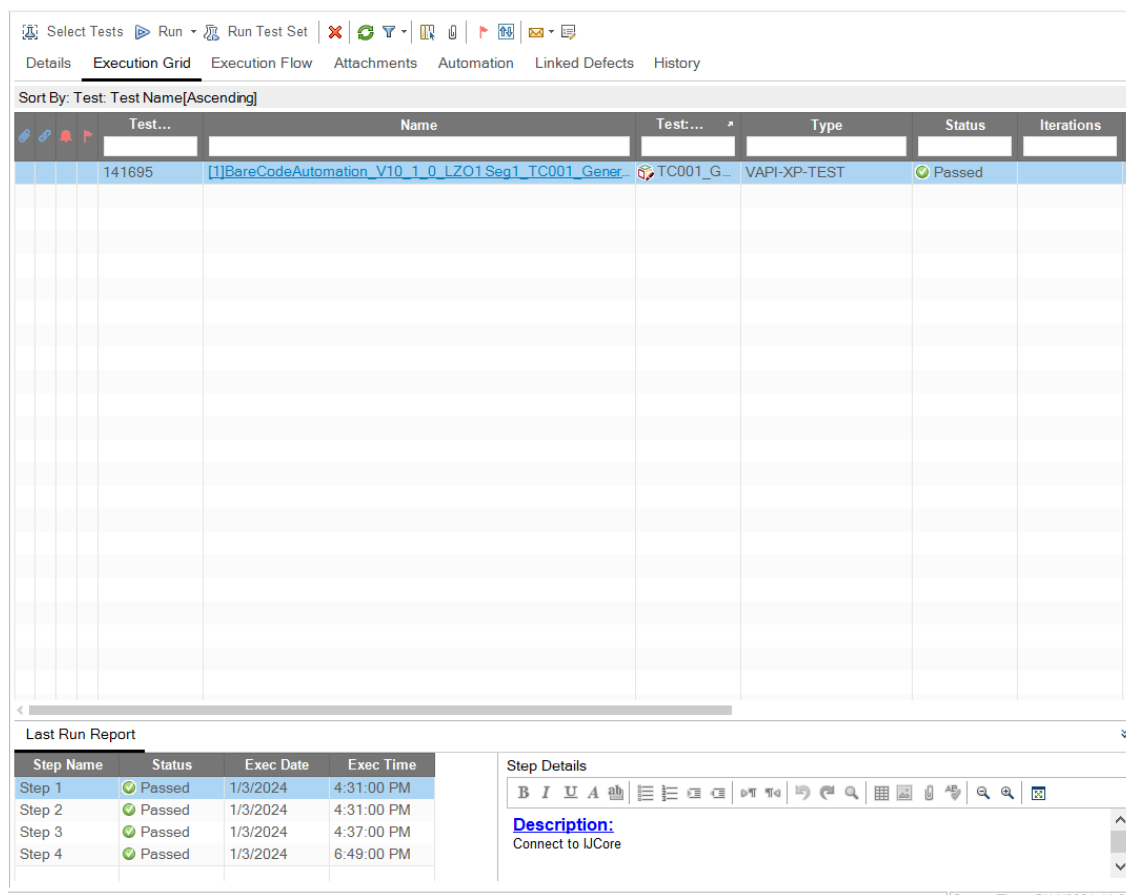
Ensuite, nous avons ajouté la configuration appropriée à la plante, en ajustant certains paramètres au besoin, tels que le nom du test, les données d'entrée, etc. comme le montre la **Figure 6.7** ci-dessous .



Used	Parameter...	Default Value	Description
	configDataFile	BarcodeConfiguration.json	
	excelReportFil...	ExcelReport.xlsx	
	FiltersDataFile	Filters.json	
	ModuleDescrip...	ModuleCode.json	
	dynamicTestC...		
	dynamicTestC...		
	dataFileName	Barcode_Generic.xlsx	
	testName	TC001_Generic_Barcode_Check_For_CustomerOrd...	
	tc_data_id		

FIGURE 6.7 : la configuration appropriée à la plante

Enfin, nous avons ajouté un ensemble de tests au niveau du laboratoire de tests QC ALM et avons procédé à l'exécution des tests comme le montre les **Figures** ci-dessous .



Test...	Name	Test:...	Type	Status	Iterations
141695	[1]BareCodeAutomation_V10_1_0_LZO1Seg1_TC001_Gener...	TC001_G...	VAPI-XP-TEST	Passed	

Step Name	Status	Exec Date	Exec Time
Step 1	Passed	1/3/2024	4:31:00 PM
Step 2	Passed	1/3/2024	4:31:00 PM
Step 3	Passed	1/3/2024	4:37:00 PM
Step 4	Passed	1/3/2024	6:49:00 PM

Description:
Connect to IJCore

Il y a une génération d'un rapport après l'exécution du test, un rapport détaillé. Si le test échoue, nous aidons à corriger les éventuelles erreurs et à affiner le processus pour améliorer

l'efficacité et la fiabilité de l'application. comme le montre la **Figure 6.8** et la **Figure 6.9**.

Barcode : CCC_REFERENCE, Module : 2BM, Calloff number : 1			1
1/3/2024 10:56:00 AM	1/3/2024 10:58:51 AM	0h:2m:50s+798ms	2
STATUS	TIMESTAMP	DETAILS	
i	10:56:00	Start checking barcode : CCC_REFERENCE	
i	10:56:00	Order : 1	
i	10:56:01	Browser navigated to the url [http://10.223.16.132//reporting/bomhistory].	
i	10:57:09	Data found on boom history. Part number saved successfully : 134B555BM	
i	10:57:10	Browser navigated to the url [http://10.223.16.132//masterdata/feature].	
i	10:57:18	Displayed table has been stabilized and it's rowcount is equal to 2	
i	10:57:23	Value of the configuration CHINA MARKET has been saved : 4M0134B555BM	
i	10:58:51		
i	10:58:51	////////////////////////////////////	
i	10:58:51	Barcode found in IJcore Customer Order Interface is : 4M0134B555BM	
i	10:58:51	Barcode Generated by the automation is : 4M0134B555BM	
✓	10:58:51	Generated barcode has the same value as the expected Barcode	3

FIGURE 6.8 : Rapport suite à l'exécution réussie du test

1 : Nom du test

2 : Temps d'exécution du test

3 : Point de vérification : Le test a été exécuté avec succès

4 : Point de vérification (échec) : Le test a échoué.

i	17:06:35	Order : 2
i	17:06:35	Order : 3
i	17:06:36	Browser navigated to the url [http://10.223.16.132//translation/translationconfiguration].
i	17:06:40	From value found 2024
i	17:06:40	ToValue has been saved and is equal to : l
i	17:06:40	Order : 4
i	17:06:40	Order : 5
i	17:06:40	Order : 6
✖	17:06:40	GetExpectedBarcodeValueForCustomerOrder crashed : Exception : Object reference not set to an instance of an object.

Test FAILED!
Message:

4

System.NullReferenceException : Object reference not set to an instance of an object.

FIGURE 6.9 : Rapport suite à l'exécution infructueuse du test

Conclusion

Dans ce chapitre, nous avons terminé la dernière partie de notre projet, durant laquelle nous avons effectué l'exécution des tests avec QC ALM et la génération des rapports.

Conclusion générale

Ce projet de fin d'études, réalisé lors de notre stage de fin d'études chez Tessian Group, avait pour objectif la mise en place d'un framework de tests automatisés et le développement des tests associés. De plus, nous avons créé un pipeline CI/CD pour déployer ces tests dans l'application de gestion du cycle de vie des applications (ALM).

Dans ce rapport, nous avons d'abord introduit le contexte général de notre projet. Ensuite, nous avons détaillé la méthodologie appliquée pour une gestion et une organisation efficaces de notre projet. Nous avons ensuite identifié les différentes exigences et besoins, puis planifié notre travail en utilisant la méthodologie Scrum, en menant une étude spécifique de chaque sprint réalisé.

En termes de perspectives, nous envisageons d'automatiser le déploiement des artefacts dans ALM, plutôt que de le faire manuellement, et d'utiliser l'intelligence artificielle pour créer les scripts de test.

Pour conclure, cette expérience chez Tessian Group nous a été très bénéfique. Elle nous a permis d'appliquer et d'approfondir les connaissances acquises durant notre cursus à Esprit, de faire le lien entre la théorie et la pratique, et de gérer un projet de manière méthodique et organisée tout en résolvant les problèmes rencontrés avec pertinence. Ce projet a donc été une source d'enrichissement tant professionnel que personnel.

Bibliographie

[1] : <https://fr.linkedin.com/company/tessangroup>

[3] : "https://dotnet.microsoft.com/en-us/"

[3] : "https://www.selenium.dev/"

[4] : <https://nunit.org/>

[5] : "https://azure.microsoft.com/en-us/products/devops/server"

[6] : "https://content.microfocus.com/alm-quality-center-16"

يندرج هذا المشروع ضمن إطار مشروع نهاية الدراسات الذي يُنفذ في شركة Tessan Group، بهدف الحصول على شهادة الهندسة الوطنية في علوم الحاسوب من الجامعة الخاصة بالهندسة والتكنولوجيا (ESPRIT). هدف هذا المشروع هو إنشاء إطار عمل لأتمتة الاختبارات وتنفيذ سلسلة للتكامل والنشر المستمر باستخدام Azure DevOps.

كلمات مفاتيح : التلقين الآلي، DevOps، ضمان الجودة والاختبارات

Résumé

Ce projet s'inscrit dans le cadre du projet de fin d'études réalisé au sein de l'entreprise Tessan Group, en vue de l'obtention du diplôme national d'ingénieur en informatique à l'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT). L'objectif de ce projet consiste à mettre en place un framework d'automatisation des testes et à implémenter un pipeline CI/CD avec Azure DevOps.

Mots clés : automatisation, DevOps, assurance qualité et tests

Abstract

This project is part of the final year project carried out within Tessan Group, aimed at obtaining the national engineering degree in computer science from the Private Higher School of Engineering and Technology (ESPRIT). The objective of this project is to set up an automation testing framework and implement a CI/CD pipeline using Azure DevOps

Keywords : Automation, DevOps, quality assurance, and testing



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454